

Contents

1	Tyrian Ledger - Qwen/MTPLX Development Package	10
1.1	Purpose	10
1.2	Development environment	11
1.3	Important editor note	11
1.4	Deliverables in this package	11
1.5	How to use this package	11
1.6	Normative language	11
2	Project Specification - Tyrian Ledger	12
2.1	1. Product statement	12
2.2	2. Product principles	12
2.3	3. Scope	12
2.3.1	MVP scope	12
2.3.2	Explicitly out of MVP scope	12
2.4	4. Users	13
2.5	5. Main user journeys	13
2.5.1	Journey A - Market flip scan	13
2.5.2	Journey B - Account-aware crafting	13
2.5.3	Journey C - Session plan	13
2.6	6. Definitions	13
2.6.1	Profit	13
2.6.2	Realized profit	13
2.6.3	Unrealized P/L	13
2.6.4	Liquidity	13
2.6.5	Confidence	14
2.7	7. Non-functional requirements	14
2.8	8. API verification policy	14
2.9	9. Market analysis requirements	14
2.9.1	Top-of-book	14
2.9.2	Order-book analysis	14
2.9.3	Fee configuration	14
2.9.4	Data freshness	14
2.10	10. Crafting requirements	15
2.11	11. Session planning requirements	15
2.12	12. History requirements	15
2.13	13. Future historical investment feature	15
2.14	14. Future LLM boundary	15
2.15	15. Completion standard	15
3	Architecture	16
3.1	Target stack	16
3.2	Runtime topology	16
3.3	Layers	16
3.3.1	Domain	16
3.3.2	Application	16
3.3.3	Infrastructure	16
3.3.4	Analytics	16
3.3.5	Web	16
3.4	Recommended repository structure	16
3.5	External API gateway	17
3.6	API caching	17
3.7	Rate-limit strategy	17
3.8	Persistence	17
3.9	Money	17
3.10	Secrets	18
3.11	Local network binding	18

3.12	Error taxonomy	18
3.13	Observability	18
3.14	LLM separation	18
4	Security and Data Protection Baseline	18
4.1	Scope	18
4.2	Mandatory/conditional requirements	19
4.3	Best practices required by this project	19
4.4	API token specifics	19
4.5	Data minimization	19
4.6	Threat model	19
4.7	Read-only enforcement	19
5	Testing Strategy	20
5.1	Quality layers	20
5.1.1	Unit tests	20
5.1.2	Integration tests	20
5.1.3	Contract/smoke tests	20
5.1.4	Browser tests	20
5.2	Test fixture policy	20
5.3	Specification-test coupling	21
5.4	Financial correctness gates	21
5.5	LLM testing	21
6	UI/UX Specification	21
6.1	Visual direction	21
6.2	Main screens	21
6.2.1	Dashboard	21
6.2.2	Opportunity detail	22
6.2.3	Crafting detail	22
6.2.4	Account/settings	22
6.3	Interaction rules	22
6.4	Desktop-first	22
7	Qwen + MTPLX Development Workflow	22
7.1	Role of Qwen	22
7.2	Context hierarchy	22
7.3	Ticket loop	23
7.4	When Qwen should stop	23
7.5	Prompt style	23
7.6	Git branch, commit, and pull-request protocol	23
7.7	Ticket delivery gate	24
7.8	ADRs	24
7.9	Review rules	24
8	Permanent Context for Qwen	24
8.1	Identity	24
8.2	Hard constraints	24
8.3	Target stack	25
8.4	Local development	25
8.5	Current architecture	25
8.6	Model/runtime note	25
8.7	Required behavior	25
9	Qwen Git and PR Context	25
9.1	Branch	25
9.2	Commits	25
9.3	Pull requests	25

10 Roadmap	26
11 ADR-001 - Application Stack	26
11.1 Status	26
11.2 Decision	26
11.3 Rationale	26
11.4 Alternative	26
12 ADR-002 - Local-First Deployment	27
12.1 Status	27
12.2 Decision	27
12.3 Consequences	27
13 ADR-003 - No Application LLM in Current Scope	27
13.1 Status	27
13.2 Decision	27
13.3 Rationale	27
13.4 Future	27
14 ADR-004 - Single GW2 API Gateway	27
14.1 Status	27
14.2 Decision	27
14.3 Rationale	27
15 ADR-005 - Integer Copper Money	28
15.1 Status	28
15.2 Decision	28
15.3 Rationale	28
16 ADR-006 - Local Secret Storage	28
16.1 Status	28
16.2 Decision	28
16.3 Development fallback	28
17 ADR-007 - Read-Only External Boundary	28
17.1 Status	28
17.2 Decision	28
17.3 Rationale	28
18 Milestone and Ticket Index	28
19 M0 - Discovery and external-contract validation	29
19.1 Goal	29
19.2 Dependencies	29
19.3 Completion rule	29
20 M1 - Repository and development foundation	29
20.1 Goal	29
20.2 Dependencies	29
20.3 Completion rule	29
21 M2 - GW2 data gateway and caching	29
21.1 Goal	29
21.2 Dependencies	30
21.3 Completion rule	30
22 M3 - Deterministic market engine	30
22.1 Goal	30
22.2 Dependencies	30

22.3	Completion rule	30
23	M4 - Dashboard and session planning	30
23.1	Goal	30
23.2	Dependencies	30
23.3	Completion rule	30
24	M5 - Account-aware analysis and crafting	30
24.1	Goal	30
24.2	Dependencies	30
24.3	Completion rule	31
25	M6 - Personal history and reconciliation	31
25.1	Goal	31
25.2	Dependencies	31
25.3	Completion rule	31
26	M7 - Historical market data and investment research	31
26.1	Goal	31
26.2	Dependencies	31
26.3	Completion rule	31
27	M8 - Hardening, accessibility, release readiness	31
27.1	Goal	31
27.2	Dependencies	31
27.3	Completion rule	31
28	TKT-M0-01 - Verify MTPLX/Qwen3.8-27B development model path	32
28.1	Milestone	32
28.2	Goal	32
28.3	Dependencies	32
28.4	Acceptance criteria	32
28.5	Required tests	32
28.6	Non-goals	32
28.7	Specification references	32
28.8	Agent prompt	32
29	TKT-M0-02 - Validate GW2 API endpoint and permission matrix	32
29.1	Milestone	32
29.2	Goal	32
29.3	Dependencies	33
29.4	Acceptance criteria	33
29.5	Required tests	33
29.6	Non-goals	33
29.7	Specification references	33
29.8	Agent prompt	33
30	TKT-M0-03 - Validate API quotas, caching guidance, and error behavior	33
30.1	Milestone	33
30.2	Goal	33
30.3	Dependencies	33
30.4	Acceptance criteria	33
30.5	Required tests	34
30.6	Non-goals	34
30.7	Specification references	34
30.8	Agent prompt	34
31	TKT-M0-04 - Validate legal/security scope and read-only boundary	34
31.1	Milestone	34

31.2	Goal	34
31.3	Dependencies	34
31.4	Acceptance criteria	34
31.5	Required tests	34
31.6	Non-goals	34
31.7	Specification references	34
31.8	Agent prompt	35
32	TKT-M1-01 - Initialize repository and solution skeleton	35
32.1	Milestone	35
32.2	Goal	35
32.3	Dependencies	35
32.4	Acceptance criteria	35
32.5	Required tests	35
32.6	Non-goals	35
32.7	Specification references	35
32.8	Agent prompt	35
33	TKT-M1-02 - Add local configuration and secret-store abstraction	35
33.1	Milestone	35
33.2	Goal	35
33.3	Dependencies	36
33.4	Acceptance criteria	36
33.5	Required tests	36
33.6	Non-goals	36
33.7	Specification references	36
33.8	Agent prompt	36
34	TKT-M1-03 - Add test infrastructure and fixture conventions	36
34.1	Milestone	36
34.2	Goal	36
34.3	Dependencies	36
34.4	Acceptance criteria	36
34.5	Required tests	36
34.6	Non-goals	37
34.7	Specification references	37
34.8	Agent prompt	37
35	TKT-M1-04 - Add local-only runtime and baseline security middleware	37
35.1	Milestone	37
35.2	Goal	37
35.3	Dependencies	37
35.4	Acceptance criteria	37
35.5	Required tests	37
35.6	Non-goals	37
35.7	Specification references	37
35.8	Agent prompt	37
36	TKT-M2-01 - Implement typed GW2 API client for public market data	38
36.1	Milestone	38
36.2	Goal	38
36.3	Dependencies	38
36.4	Acceptance criteria	38
36.5	Required tests	38
36.6	Non-goals	38
36.7	Specification references	38
36.8	Agent prompt	38

37 TKT-M2-02 - Build request scheduler, deduplication, and retry policy	38
37.1 Milestone	38
37.2 Goal	38
37.3 Dependencies	39
37.4 Acceptance criteria	39
37.5 Required tests	39
37.6 Non-goals	39
37.7 Specification references	39
37.8 Agent prompt	39
38 TKT-M2-03 - Implement cache with freshness metadata	39
38.1 Milestone	39
38.2 Goal	39
38.3 Dependencies	39
38.4 Acceptance criteria	39
38.5 Required tests	40
38.6 Non-goals	40
38.7 Specification references	40
38.8 Agent prompt	40
39 TKT-M2-04 - Add market-data observability and diagnostics	40
39.1 Milestone	40
39.2 Goal	40
39.3 Dependencies	40
39.4 Acceptance criteria	40
39.5 Required tests	40
39.6 Non-goals	40
39.7 Specification references	40
39.8 Agent prompt	41
40 TKT-M3-01 - Implement Money and fee policy	41
40.1 Milestone	41
40.2 Goal	41
40.3 Dependencies	41
40.4 Acceptance criteria	41
40.5 Required tests	41
40.6 Non-goals	41
40.7 Specification references	41
40.8 Agent prompt	41
41 TKT-M3-02 - Implement order-book quantity simulation	41
41.1 Milestone	41
41.2 Goal	41
41.3 Dependencies	42
41.4 Acceptance criteria	42
41.5 Required tests	42
41.6 Non-goals	42
41.7 Specification references	42
41.8 Agent prompt	42
42 TKT-M3-03 - Implement flip profitability and liquidity analysis	42
42.1 Milestone	42
42.2 Goal	42
42.3 Dependencies	42
42.4 Acceptance criteria	42
42.5 Required tests	43
42.6 Non-goals	43
42.7 Specification references	43

42.8 Agent prompt	43
43 TKT-M3-04 - Implement deterministic opportunity scoring	43
43.1 Milestone	43
43.2 Goal	43
43.3 Dependencies	43
43.4 Acceptance criteria	43
43.5 Required tests	43
43.6 Non-goals	43
43.7 Specification references	44
43.8 Agent prompt	44
44 TKT-M4-01 - Create market dashboard and filters	44
44.1 Milestone	44
44.2 Goal	44
44.3 Dependencies	44
44.4 Acceptance criteria	44
44.5 Required tests	44
44.6 Non-goals	44
44.7 Specification references	44
44.8 Agent prompt	44
45 TKT-M4-02 - Create opportunity detail view	45
45.1 Milestone	45
45.2 Goal	45
45.3 Dependencies	45
45.4 Acceptance criteria	45
45.5 Required tests	45
45.6 Non-goals	45
45.7 Specification references	45
45.8 Agent prompt	45
46 TKT-M4-03 - Implement user session preferences	45
46.1 Milestone	45
46.2 Goal	45
46.3 Dependencies	45
46.4 Acceptance criteria	46
46.5 Required tests	46
46.6 Non-goals	46
46.7 Specification references	46
46.8 Agent prompt	46
47 TKT-M4-04 - Implement session planner with coarse effort categories	46
47.1 Milestone	46
47.2 Goal	46
47.3 Dependencies	46
47.4 Acceptance criteria	46
47.5 Required tests	46
47.6 Non-goals	46
47.7 Specification references	47
47.8 Agent prompt	47
48 TKT-M5-01 - Implement token validation and permission-aware account access	47
48.1 Milestone	47
48.2 Goal	47
48.3 Dependencies	47
48.4 Acceptance criteria	47
48.5 Required tests	47

48.6 Non-goals	47
48.7 Specification references	47
48.8 Agent prompt	47
49 TKT-M5-02 - Implement account snapshot ingestion	48
49.1 Milestone	48
49.2 Goal	48
49.3 Dependencies	48
49.4 Acceptance criteria	48
49.5 Required tests	48
49.6 Non-goals	48
49.7 Specification references	48
49.8 Agent prompt	48
50 TKT-M5-03 - Implement owned-item opportunity cost	48
50.1 Milestone	48
50.2 Goal	48
50.3 Dependencies	48
50.4 Acceptance criteria	49
50.5 Required tests	49
50.6 Non-goals	49
50.7 Specification references	49
50.8 Agent prompt	49
51 TKT-M5-04 - Implement bounded crafting path search	49
51.1 Milestone	49
51.2 Goal	49
51.3 Dependencies	49
51.4 Acceptance criteria	49
51.5 Required tests	49
51.6 Non-goals	50
51.7 Specification references	50
51.8 Agent prompt	50
52 TKT-M6-01 - Create operation history model and local persistence	50
52.1 Milestone	50
52.2 Goal	50
52.3 Dependencies	50
52.4 Acceptance criteria	50
52.5 Required tests	50
52.6 Non-goals	50
52.7 Specification references	50
52.8 Agent prompt	51
53 TKT-M6-02 - Implement realized profit reconciliation	51
53.1 Milestone	51
53.2 Goal	51
53.3 Dependencies	51
53.4 Acceptance criteria	51
53.5 Required tests	51
53.6 Non-goals	51
53.7 Specification references	51
53.8 Agent prompt	51
54 TKT-M6-03 - Build personal dashboard/history statistics	51
54.1 Milestone	51
54.2 Goal	52
54.3 Dependencies	52

54.4	Acceptance criteria	52
54.5	Required tests	52
54.6	Non-goals	52
54.7	Specification references	52
54.8	Agent prompt	52
55	TKT-M6-04 - Implement local data clearing and backup guidance	52
55.1	Milestone	52
55.2	Goal	52
55.3	Dependencies	52
55.4	Acceptance criteria	52
55.5	Required tests	53
55.6	Non-goals	53
55.7	Specification references	53
55.8	Agent prompt	53
56	TKT-M7-01 - Design local market snapshot schema and sampling policy	53
56.1	Milestone	53
56.2	Goal	53
56.3	Dependencies	53
56.4	Acceptance criteria	53
56.5	Required tests	53
56.6	Non-goals	53
56.7	Specification references	53
56.8	Agent prompt	54
57	TKT-M7-02 - Implement economical historical collection scheduler	54
57.1	Milestone	54
57.2	Goal	54
57.3	Dependencies	54
57.4	Acceptance criteria	54
57.5	Required tests	54
57.6	Non-goals	54
57.7	Specification references	54
57.8	Agent prompt	54
58	TKT-M7-03 - Implement descriptive historical analytics	54
58.1	Milestone	54
58.2	Goal	54
58.3	Dependencies	55
58.4	Acceptance criteria	55
58.5	Required tests	55
58.6	Non-goals	55
58.7	Specification references	55
58.8	Agent prompt	55
59	TKT-M7-04 - Create investment-research view with uncertainty	55
59.1	Milestone	55
59.2	Goal	55
59.3	Dependencies	55
59.4	Acceptance criteria	55
59.5	Required tests	55
59.6	Non-goals	56
59.7	Specification references	56
59.8	Agent prompt	56
60	TKT-M8-01 - Perform security review and secret-leak audit	56
60.1	Milestone	56

60.2	Goal	56
60.3	Dependencies	56
60.4	Acceptance criteria	56
60.5	Required tests	56
60.6	Non-goals	56
60.7	Specification references	56
60.8	Agent prompt	56
61	TKT-M8-02 - Perform browser/accessibility and UX hardening	57
61.1	Milestone	57
61.2	Goal	57
61.3	Dependencies	57
61.4	Acceptance criteria	57
61.5	Required tests	57
61.6	Non-goals	57
61.7	Specification references	57
61.8	Agent prompt	57
62	TKT-M8-03 - Performance and API-request efficiency review	57
62.1	Milestone	57
62.2	Goal	57
62.3	Dependencies	57
62.4	Acceptance criteria	58
62.5	Required tests	58
62.6	Non-goals	58
62.7	Specification references	58
62.8	Agent prompt	58
63	TKT-M8-04 - Prepare local release package and handoff documentation	58
63.1	Milestone	58
63.2	Goal	58
63.3	Dependencies	58
63.4	Acceptance criteria	58
63.5	Required tests	58
63.6	Non-goals	59
63.7	Specification references	59
63.8	Agent prompt	59

1 Tyrian Ledger - Qwen/MTPLX Development Package

This repository is the complete planning and development package for a local, read-only Guild Wars 2 Trading Post analysis application.

1.1 Purpose

Build a browser-based local application that:

- reads Guild Wars 2 public market data and, optionally, account data through the GW2 API;
- minimizes API requests through batching, caching, deduplication, and scheduled refreshes;
- never automates gameplay, Trading Post actions, or API mutations;
- calculates market and crafting opportunities deterministically;
- makes assumptions, data age, uncertainty, liquidity, and opportunity cost visible;
- persists local user history without requiring a server or cloud account;
- can later support historical analysis and long-term investment research;
- may later add an application-facing LLM, but **no LLM integration is part of the MVP or current architecture.**

1.2 Development environment

The local coding agent is Qwen3.8-27B operated through MTPLX on an Apple Silicon Mac. MTPLX exposes an OpenAI-compatible local API and is used only as the development assistant/runtime. It is not an application runtime dependency.

Important: MTPLX currently runs MLX-native model artifacts and explicitly treats GGUF as the llama.cpp format. The raw `unsloth/Qwen3.8-27B-GGUF` repository must therefore be treated as a source model reference, not as a guaranteed MTPLX-loadable artifact. Milestone M0 contains an explicit compatibility gate before development proceeds.

1.3 Important editor note

Visual Studio for Mac was retired on August 31, 2024. On macOS use Visual Studio Code (or another current editor) with the local MTPLX-compatible coding workflow.

1.4 Deliverables in this package

- `docs/specs/project-spec.md` - normative product and system specification.
- `docs/architecture/architecture.md` - technical architecture.
- `docs/security/security.md` - security and French/EU data-protection baseline.
- `docs/testing/testing-strategy.md` - test strategy and quality gates.
- `docs/ux/ux.md` - UI/UX rules.
- `docs/workflow/ai-development-workflow.md` - development workflow for Qwen via MTPLX.
- `docs/adr/` - Architecture Decision Records (ADRs).
- `docs/milestones/` - milestone definitions and completion criteria.
- `docs/context/` - lightweight context files intended for Qwen, including Git/PR delivery rules.
- `docs/tickets/` - implementation tickets.
- `docs/prompts/` - one execution prompt per ticket.
- `config/` - repository configuration templates.
- `.github/pull_request_template.md` - standard ticket PR structure.

1.5 How to use this package

1. Create an empty Git repository.
2. Copy the contents of this package into it.
3. Read `docs/specs/project-spec.md` once yourself.
4. Load only `docs/context/permanent-context.md`, the current milestone context, and one ticket into Qwen at a time.
5. Execute tickets in numerical order within each milestone unless an explicit dependency says otherwise.
6. Require tests for every behavior change.
7. Do not let Qwen silently change architectural decisions. If a ticket exposes a real architectural issue, update or create an ADR first.
8. Keep the application fully read-only and local until a future scope explicitly changes that decision.
9. For every ticket, use a dedicated branch, prefix every ticket commit with `[TICKET_NAME]`, create a GitHub PR titled `[TICKET_NAME] Short title`, include exact specification references in the PR body, and never merge the PR as Qwen.

1.6 Normative language

- MUST = mandatory.
- MUST NOT = prohibited.
- SHOULD = strong recommendation; deviation requires rationale.
- MAY = optional.
- ASSUMPTION = provisional until verified.
- VERIFY = must be checked against current authoritative documentation or measured behavior before release.

2 Project Specification - Tyrian Ledger

2.1 1. Product statement

The system is a local, browser-based decision-support tool for Guild Wars 2 Trading Post and crafting analysis. Its purpose is to help a single user identify potentially profitable in-game gold-making opportunities using legal, read-only analysis.

The application is not a trading bot, gameplay bot, order executor, browser automator, or autonomous game agent.

2.2 2. Product principles

1. **Read-only by design.** The application may request permitted read-only GW2 API resources. It MUST NOT automate Trading Post actions or gameplay.
2. **Deterministic financial truth.** Money calculations, fee calculations, opportunity ranking, and crafting feasibility are performed by deterministic application code. A future LLM MAY explain results but MUST NOT become the source of numerical truth.
3. **Data freshness is first-class.** Every market-derived result carries capture time and a data-quality/freshness state.
4. **API minimization.** All GW2 requests go through one server-side gateway with caching, batching, deduplication, scheduling, and retry/rate-limit handling.
5. **Explicit assumptions.** A result is a scenario, not a guarantee of execution or profit.
6. **Opportunity cost matters.** Owned items are never treated as free inputs by default.
7. **Local-first.** No server, account system, cloud database, or mandatory remote service is required for v1.
8. **Testability.** Core calculations are pure and independently testable.
9. **Secure by default.** The API key stays server-side/local and outside version control.
10. **Incremental complexity.** Features requiring historical data are deferred until local collection has created enough evidence.

2.3 3. Scope

2.3.1 MVP scope

- current market price and order-book ingestion;
- caching and request scheduling;
- fee-aware flipping scenarios;
- order-book depth simulation;
- opportunity ranking using deterministic scoring;
- user constraints: capital, strategy preference, minimum profit, risk tolerance, capital allocation, data freshness;
- local account connection with minimum required API permissions;
- account data snapshots for bank/materials and relevant character crafting information;
- limited-depth crafting profitability;
- owned-material opportunity-cost analysis;
- session planning using coarse effort categories;
- modern desktop browser UI;
- comprehensive unit/integration/fixture tests;
- local SQLite persistence;
- clear read-only disclosures and data freshness indicators.

2.3.2 Explicitly out of MVP scope

- any action that places/cancels TP orders;
- gameplay automation;
- browser automation against the game or TP;
- a public/multi-user server;
- cloud persistence;
- mandatory LLM integration;
- precise execution-time prediction;
- price forecasting marketed as reliable prediction;

- long-term investment recommendations based only on snapshots;
- unrestricted full-world crafting path optimization;
- automatic profit reconciliation with in-game state beyond available read-only transaction/account data;
- mobile-first layouts.

2.4 4. Users

Initial user: one human developer/player operating the tool locally on their own Mac.

No user-to-user sharing is required for v1.

2.5 5. Main user journeys

2.5.1 Journey A - Market flip scan

1. User opens the local dashboard.
2. Application shows when market data was last refreshed.
3. User chooses capital and risk/strategy preferences.
4. Application selects candidate items from available market data.
5. For each candidate the deterministic engine calculates scenario profit, ROI, capital need, price impact, liquidity proxy, and data freshness.
6. UI ranks and filters opportunities.
7. User opens an opportunity to see assumptions and calculation details.

2.5.2 Journey B - Account-aware crafting

1. User supplies an API key with the minimum necessary scopes.
2. Backend validates the token and reports available permissions without exposing the secret.
3. Account snapshots are fetched only when needed.
4. Crafting analyzer evaluates feasible recipes.
5. Owned materials are valued economically rather than at zero.
6. UI compares buy-all, use-owned, and mixed strategies when data permits.
7. User sees the exact intermediate path used by the calculation.

2.5.3 Journey C - Session plan

1. User chooses available capital, preferred strategy, desired risk, and an effort category.
2. Engine filters opportunities that do not fit.
3. Planner builds a ranked set without claiming exact execution time.
4. User can save selected opportunities to local history.

2.6 6. Definitions

2.6.1 Profit

Profit is the modeled net economic gain for a specific scenario after explicitly modeled transaction fees and acquisition costs.

2.6.2 Realized profit

Once history exists, realized profit is based only on recorded actual acquisition/sale values and actual applicable fees, not current market value.

2.6.3 Unrealized P/L

A separate statistic comparing current modeled value to recorded cost basis. It MUST NOT be mixed with realized profit.

2.6.4 Liquidity

A proxy derived from current order-book depth and price impact. It is not a guaranteed execution probability.

2.6.5 Confidence

A qualitative or component-based indication of evidence quality. It MUST NOT be represented as a mathematically precise probability unless a validated statistical model exists.

2.7 7. Non-functional requirements

- Local application starts without an external server.
- Default bind address MUST be loopback-only.
- No API key is sent to browser JavaScript.
- No API key is written to logs.
- Core calculations are deterministic for identical inputs and configuration.
- Tests MUST run without live GW2 API access.
- UI MUST support current major desktop browsers. Exact browser matrix is a release configuration item.
- Accessibility: keyboard navigation, readable contrast, semantic controls, sensible focus management.
- No decorative reuse of GW2 proprietary UI assets.

2.8 8. API verification policy

Exact endpoint schemas, scopes, current quotas, cache guidance, and field meanings are external contracts and MUST be revalidated before release.

Initial authoritative/community references to verify:

- <https://wiki.guildwars2.com/wiki/API:2/commerce/prices>
- <https://wiki.guildwars2.com/wiki/API:2/commerce/listings>
- <https://wiki.guildwars2.com/wiki/API:2/commerce/transactions>
- <https://wiki.guildwars2.com/wiki/API:2/recipes>
- <https://wiki.guildwars2.com/wiki/API:2/recipes/search>
- <https://wiki.guildwars2.com/wiki/API:2/account/bank>
- <https://wiki.guildwars2.com/wiki/API:2/account/materials>
- <https://wiki.guildwars2.com/wiki/API:2/characters/:id/crafting>
- <https://wiki.guildwars2.com/wiki/API:2/account/recipes>
- <https://wiki.guildwars2.com/wiki/API:2/tokeninfo>
- https://wiki.guildwars2.com/wiki/API:Best_practices
- https://wiki.guildwars2.com/wiki/API:Terms_of_Use

If an exact field, quota, fee behavior, or permission is uncertain, the implementation MUST mark it VERIFY and stop short of treating the assumption as authoritative.

2.9 9. Market analysis requirements

2.9.1 Top-of-book

The engine MAY use current aggregate prices for cheap broad candidate screening.

2.9.2 Order-book analysis

Detailed listings SHOULD be used for candidates that survive screening. The engine SHOULD simulate quantity-aware acquisition and liquidation across multiple price levels.

2.9.3 Fee configuration

Fee rules MUST be centralized in a configurable policy. The application MUST NOT scatter magic constants through analytical code.

2.9.4 Data freshness

Thresholds MUST be configurable. Suggested initial states are Fresh, Aging, Stale, Incomplete. These labels are product policy, not API facts.

2.10 10. Crafting requirements

The analyzer **MUST** respect available recipes, disciplines/ratings, recipe availability, and ingredient quantities when those facts are accessible.

For a multi-step path the engine **MUST**:

- detect cycles;
- impose a configurable maximum depth;
- memoize repeated subproblems;
- stop or truncate gracefully when the search space becomes too large;
- expose when analysis was truncated;
- compare purchased ingredient economics against owned-item opportunity cost.

2.11 11. Session planning requirements

The initial planner uses categories rather than exact execution promises:

- Very low effort
- Low effort
- Medium effort
- High effort
- Ongoing/patient

The user **MAY** override category preferences. Precise time estimates require later data collection and calibration.

2.12 12. History requirements

Local persistence **MAY** include:

- user preference profile;
- saved opportunities;
- planned operations;
- recorded actual outcomes;
- locally captured market snapshots;
- calculation versions/strategy configuration identifiers;
- account snapshots, with data minimization.

The UI **MUST** let the user clear local account-related data.

2.13 13. Future historical investment feature

The application **MAY** collect sampled market snapshots locally. Historical analysis should not begin as a predictive feature.

First provide descriptive measures such as observed percentile, volatility, spread persistence, drawdown, and liquidity stability. Only later consider predictive strategies, with explicit validation against out-of-sample data.

2.14 14. Future LLM boundary

No LLM integration is required for the current project. If later introduced, it **MUST** be an adapter around deterministic application services and **MUST NOT** have direct access to the GW2 API key or authority to perform mutations.

2.15 15. Completion standard

A release is not complete because the UI looks correct. A release is complete only when:

- requirements are reflected in tests;
- tests pass from a clean checkout;
- API assumptions are verified/documented;
- no secret is committed;
- rate-limit/caching behavior is observable;
- calculations are reproducible;

- read-only constraints are demonstrably enforced by architecture.
-

3 Architecture

3.1 Target stack

Recommended baseline:

- .NET 10 LTS backend with ASP.NET Core
- React + TypeScript frontend
- SQLite via EF Core or a narrowly-scoped persistence abstraction
- xUnit for .NET tests
- Playwright for browser smoke/end-to-end tests
- built-in structured logging with redaction rules

Alternative: Blazor Web App if a single-language C# stack is strongly preferred. Do not switch to Blazor solely because it looks simpler on day one; the current specification benefits from a clear browser/client boundary and TypeScript ecosystem.

3.2 Runtime topology

Browser -> ASP.NET Core local server -> application services -> deterministic analytics / SQLite / GW2 API gateway

No browser component calls GW2 authenticated endpoints directly.

3.3 Layers

3.3.1 Domain

Pure models and invariants. No HTTP, database, framework, or UI dependencies.

3.3.2 Application

Use cases and orchestration. Defines interfaces such as market data provider, account data provider, snapshot repository, clock, and secret store.

3.3.3 Infrastructure

GW2 HTTP implementation, caching, persistence, secret handling, filesystem configuration, external adapters.

3.3.4 Analytics

Pure deterministic calculators: fees, market scenarios, order-book simulation, crafting path search, opportunity score, stale-data handling.

3.3.5 Web

HTTP endpoints, DTO mapping, UI, validation, presentation.

3.4 Recommended repository structure

```
/
src/
  Gw2Tp.Web/
  Gw2Tp.Application/
  Gw2Tp.Domain/
  Gw2Tp.Analytics/
  Gw2Tp.Infrastructure/
```



```
tests/  
  Gw2Tp.Domain.Tests/  
  Gw2Tp.Application.Tests/  
  Gw2Tp.Analytics.Tests/  
  Gw2Tp.Infrastructure.Tests/  
  Gw2Tp.IntegrationTests/  
  Gw2Tp.Web.E2E/  
docs/  
config/
```

3.5 External API gateway

All GW2 API calls MUST pass through one abstraction:

`IGw2ApiClient`

with internal components:

- request scheduler;
- cache;
- deduplicator;
- batching coordinator;
- retry/backoff policy;
- response validation;
- metrics/logging.

Feature code must never construct an ArenaNet URL itself.

3.6 API caching

Reference data: long TTL and explicit refresh/invalidation.

Market data: short TTL and freshness metadata.

Account snapshots: user-controlled refresh or moderate TTL.

All cached authenticated responses are scoped to a local account identity/key identifier and are never mixed across account contexts.

3.7 Rate-limit strategy

The request scheduler should support a configurable token bucket. Current documented/community limits must be verified before release, not hard-coded as immutable facts.

On 429:

1. record the event;
2. respect server-provided retry information when available;
3. apply bounded exponential backoff;
4. suppress duplicate concurrent requests;
5. surface a useful UI state.

3.8 Persistence

SQLite is the default local store. Use migrations and schema versioning.

Persist only information needed for the feature. Keep raw API snapshots only where they provide a clear debugging/historical value.

3.9 Money

Represent money in integer copper using a dedicated value type or equivalent. No floating-point arithmetic for gold/silver/copper calculations.

3.10 Secrets

Use an OS-backed local secret store through an abstraction. For development, support an environment-variable override that is explicitly documented as local development only.

3.11 Local network binding

Default server address: 127.0.0.1.

Do not expose the application to the LAN by default.

3.12 Error taxonomy

External failures should map to stable application error categories:

- AuthenticationFailed
- PermissionDenied
- RateLimited
- TemporarilyUnavailable
- NotFound
- InvalidRemoteData
- UnsupportedSchema
- LocalConfigurationError

3.13 Observability

Track:

- API request count by endpoint category;
- cache hit/miss;
- request latency;
- 429 count;
- failed parsing;
- market data age;
- analytics duration;
- number of candidates analyzed;
- search truncation events.

Never log API keys, authorization headers, or complete sensitive account payloads.

3.14 LLM separation

No application LLM in current scope. The coding model is an external development dependency. If a future LLM feature is added, it must consume structured application output through an adapter and must not access secrets or mutate state.

4 Security and Data Protection Baseline

4.1 Scope

The initial application is local and personal. This is intentionally not a multi-user web service.

For a genuinely personal/household activity, GDPR Article 2(2)(c) can exclude the activity from GDPR scope. That conclusion is context-dependent and must be reassessed if the application is commercialized, shared, hosted, or used for other persons' data.

References: - <https://eur-lex.europa.eu/eli/reg/2016/679/oj> - <https://www.cnil.fr/fr/securite-api-interfaces-de-programmation-applicative> - <https://www.cnil.fr/fr/passer-laction/garantir-la-securite-des-donnees> - <https://www.ssi.gouv.fr/entraine> pratiques/

4.2 Mandatory/conditional requirements

1. Comply with current Guild Wars 2 API and website/content terms.
2. If the scope becomes non-personal, apply GDPR/French data-protection obligations appropriate to that deployment.
3. Protect API credentials against unauthorized access.
4. Apply security controls proportionate to risk.
5. Reassess legal/security scope before public or hosted deployment.

4.3 Best practices required by this project

- API key stored outside source code;
- no secret in browser JavaScript;
- loopback-only server by default;
- redacted logs;
- minimal local data retention;
- explicit clear-data action;
- dependency updates;
- test fixtures free of real API keys;
- no analytics/telemetry sent remotely without explicit future decision;
- secure default headers where compatible with local development;
- input validation on all locally exposed HTTP endpoints;
- no unsafe HTML rendering of API-controlled names or text.

4.4 API token specifics

The current GW2 tokeninfo documentation warns that the key name is not escaped and can contain HTML/JavaScript. Render it as text only.

Permissions required by the application must be declared per feature. The key validator should show missing permissions and disable corresponding functionality instead of repeatedly attempting unauthorized calls.

4.5 Data minimization

Prefer storing:

- opaque account identifier or local profile ID;
- selected account-derived facts needed for current analysis;
- timestamps;
- user-entered preferences;
- saved operation outcomes.

Avoid storing full raw authenticated payloads indefinitely.

4.6 Threat model

Threats include:

- accidental Git secret commit;
- malicious local webpage attempting to call the local server;
- XSS from API-derived strings;
- compromised local process reading secrets;
- dependency vulnerability;
- unauthorized LAN access if binding changes;
- stale/corrupt market data causing bad decisions.

4.7 Read-only enforcement

The application should not expose an HTTP client abstraction that can issue arbitrary authenticated methods/URLs. Restrict the API adapter to a typed allow-list of known GET resources.

The project should include a regression test that no write-capable API method exists in the GW2 adapter surface.

5 Testing Strategy

5.1 Quality layers

5.1.1 Unit tests

Required for all deterministic business logic:

- Money and currency conversion
- Fee policy
- Profit calculations
- ROI and capital efficiency
- Order-book depth simulation
- Liquidity metrics
- Freshness classification
- Opportunity scoring
- Session planning
- Owned-item opportunity cost
- Crafting graph/search
- Data validation and normalization

5.1.2 Integration tests

Use recorded fixtures or mocks for GW2 responses. Normal automated tests must not call live endpoints.

Cover:

- prices
- listings
- recipes
- account tokeninfo
- bank/materials/inventory
- character crafting/recipe availability
- transaction history where used
- 401/403/404/429/5xx
- malformed JSON
- missing/new fields

5.1.3 Contract/smoke tests

A controlled, manually triggered or release-stage suite MAY call the live API to confirm endpoint shape. This is not the normal CI suite.

5.1.4 Browser tests

Use Playwright to test the highest-value journeys:

1. dashboard loads with fixture data;
2. filters work;
3. opportunity detail explains assumptions;
4. API permission failure is understandable;
5. no secret appears in HTML or network responses;
6. data freshness is visible.

5.2 Test fixture policy

Fixtures must be:

- deterministic;
- synthetic or sanitized;
- small enough to inspect manually;
- versioned with the test that depends on them.

5.3 Specification-test coupling

Every ticket that changes behavior must update at least one of:

- acceptance tests;
- unit tests;
- integration fixtures/tests;
- browser tests;
- specification text.

The PR/commit should make the change relationship obvious.

5.4 Financial correctness gates

The following cannot ship with failing tests:

- copper arithmetic;
- fee calculation;
- order-book simulation;
- crafting economic cost;
- realized profit reconciliation.

5.5 LLM testing

No LLM application tests exist in current scope. Development-agent behavior is evaluated operationally through ticket acceptance and repository tests.

If a future LLM feature is added, deterministic invariants must still be tested outside the LLM.

6 UI/UX Specification

6.1 Visual direction

Modern analytical dashboard with restrained Guild Wars 2-inspired atmosphere without copying ArenaNet assets.

Use:

- dark/charcoal base;
- subtle metallic/parchment surfaces;
- restrained cyan/teal accents;
- clear gold/copper emphasis only for economic values;
- high information density with generous grouping;
- simple icons and CSS shapes instead of copied game assets.

6.2 Main screens

6.2.1 Dashboard

- opportunity ranking;
- user/session constraints;
- data freshness;
- last refresh;
- warning banner when data is stale/incomplete.

6.2.2 Opportunity detail

Show:

- exact scenario;
- acquisition assumption;
- exit assumption;
- fees;
- modeled profit;
- capital required;
- ROI;
- order-book impact;
- liquidity proxy;
- risk/confidence factors;
- freshness.

6.2.3 Crafting detail

Show the path as a tree or expandable graph:

output -> recipe -> intermediate ingredients -> source cost.

Highlight where owned materials are used and show their opportunity cost.

6.2.4 Account/settings

- API key connection state;
- permission status;
- data refresh controls;
- stored profile settings;
- clear local account data.

6.3 Interaction rules

- Do not hide critical assumptions behind tooltips only.
- Use explicit labels: **Modeled profit**, not **Profit**.
- Use **Data age** everywhere live market data appears.
- Never show a green “guaranteed” state.
- Use filter controls that preserve current selections when possible.
- Support keyboard navigation.

6.4 Desktop-first

Mobile support is deferred. The desktop layout should be responsive enough to avoid unusable overflow at narrower desktop window widths.

7 Qwen + MTPLX Development Workflow

7.1 Role of Qwen

Qwen is the development agent. It is not a product decision-maker and is not the application’s business-logic authority.

7.2 Context hierarchy

Load the minimum context needed:

1. docs/context/permanent-context.md
2. docs/context/milestone-context-Mx.md
3. exactly one ticket from docs/tickets/

4. the ticket prompt from `docs/prompts/`
5. only the relevant source files.

Do not paste the entire specification into every request.

7.3 Ticket loop

For every ticket:

1. Read the ticket and acceptance criteria.
2. Inspect current repository state.
3. Identify contradictions or missing dependencies.
4. State implementation plan briefly.
5. Implement only the ticket scope.
6. Add/update tests before declaring success.
7. Run formatter/analyzers/tests.
8. Inspect changed files for secret leakage and accidental scope expansion.
9. Report files changed, tests run, known limitations, and any specification issue.

7.4 When Qwen should stop

Qwen must stop and ask for a human decision when:

- a current API fact contradicts the specification;
- a ticket requires a write-capable external operation;
- an architectural decision must change;
- a secret is required but no secure mechanism is configured;
- tests and requirements appear mutually inconsistent;
- a dependency requires accepting an unacceptable license or security risk.

7.5 Prompt style

Prompts should be imperative, scoped, test-oriented, and explicit about non-goals. Avoid “build the app” requests.

7.6 Git branch, commit, and pull-request protocol

Each ticket is an independently reviewable unit. Qwen MUST create a dedicated branch before implementation:

`ticket/<TICKET_NAME>-<short-kebab-title>`

All commits for that ticket MUST start with the exact ticket identifier in square brackets:

`[TKT-M3-02] Add fee-aware flip calculator`

One logical ticket per commit is preferred; multiple commits are allowed when they improve reviewability, but every commit must use the ticket prefix.

Every completed ticket MUST have a GitHub pull request. The PR title MUST use:

`[TICKET_NAME] Short title`

Example:

`[TKT-M3-02] Add fee-aware flip calculator`

The PR body MUST include:

- **Ticket:** exact ticket identifier and link/path to the ticket.
- **Milestone:** milestone identifier.
- **Specification references:** exact specification, architecture, ADR, security, testing, and/or UX documents that this PR implements or validates.
- **Summary:** concise description of what changed.
- **Acceptance criteria:** checklist showing each criterion as complete, incomplete, or not applicable with rationale.
- **Validation:** commands/tests run and their results.

- **Decisions:** ADRs or durable decisions made, if any.
- **VERIFY items:** unresolved external facts or assumptions.
- **Risks/limitations:** relevant known limitations.
- **Follow-up:** remaining work, if any.

Qwen MUST actually create the PR and report its URL. It MUST NOT invent a URL or claim that a PR exists without verifying it. If the GitHub CLI, authentication, remote, or permissions needed to create the PR are unavailable, Qwen MUST stop at the delivery gate and report the blocker rather than declaring the ticket complete.

Qwen MUST NOT merge the PR. Human review and merge are mandatory.

7.7 Ticket delivery gate

A ticket is not complete until all of the following are true:

1. Acceptance criteria are satisfied or an explicit blocker is recorded.
2. Relevant tests/validation have been run.
3. The diff has been reviewed for scope expansion and secrets.
4. All ticket commits use the required [TICKET_NAME] prefix.
5. The branch is pushed to the configured GitHub remote.
6. A PR exists with the required title and specification references.
7. Qwen reports the PR URL and does not merge it.

7.8 ADRs

An ADR is an Architecture Decision Record: a short document that records an important architectural decision, its context, the chosen option, alternatives, and consequences.

Create/update an ADR when a decision is durable and cross-cutting.

7.9 Review rules

Human review is required for:

- changes to API permissions;
- secrets/authentication;
- API request policy;
- financial formulas;
- persistence schema decisions;
- security behavior;
- additions to project scope.

8 Permanent Context for Qwen

8.1 Identity

You are the coding agent for a local Guild Wars 2 Trading Post analysis application.

8.2 Hard constraints

- Read-only application.
- No gameplay automation.
- No Trading Post automation.
- No API mutation/write operations.
- No secret in source code, browser code, logs, fixtures, prompts, or tests.
- Core financial truth must be deterministic.
- All GW2 API access goes through one typed gateway with caching/rate limiting.
- Money uses integer copper.
- Tests are required for business logic changes.

- Never invent undocumented GW2 API fields or quotas.
- Preserve modular boundaries.
- Do not add an application LLM; Qwen is only the development agent.

8.3 Target stack

- ASP.NET Core / .NET 10 LTS
- React + TypeScript
- SQLite
- xUnit
- Playwright for browser tests

8.4 Local development

macOS Apple Silicon. Visual Studio for Mac is retired; use Visual Studio Code or equivalent. MTPLX serves the local coding model on loopback.

8.5 Current architecture

Browser -> Web API -> Application services -> Analytics/Infrastructure -> SQLite/GW2 API.

8.6 Model/runtime note

MTPLX is MLX-native. The raw `unsloth/Qwen3.8-27B-GGUF` model is not assumed to load directly. Compatibility must be validated in M0. The application itself must not depend on MTPLX.

8.7 Required behavior

When uncertain, mark `VERIFY` and avoid inventing facts.

Prefer small, reversible changes.

9 Qwen Git and PR Context

This is a lightweight delivery context. Read it for every ticket.

9.1 Branch

Create one branch per ticket:

`ticket/<TICKET_NAME>-<short-kebab-title>`

9.2 Commits

Every commit for the ticket **MUST** start with:

`[TICKET_NAME]`

Example: `[TKT-M0-01] Record MTPLX model compatibility decision`

9.3 Pull requests

Every ticket **MUST** produce one GitHub pull request before it can be declared complete.

PR title format:

`[TICKET_NAME] Short title`

The PR body must identify the ticket, milestone, exact specification/architecture/ADR/testing/security/UX references, summary, acceptance criteria, validation, decisions, `VERIFY` items, risks/limitations, and follow-up work.

Qwen must verify the PR exists and report its URL. Never invent a PR URL. Never merge the PR; human review and merge are required.

If GitHub remote access, authentication, permissions, or the required CLI are unavailable, stop at the delivery gate and report the blocker instead of declaring the ticket complete.

10 Roadmap

Phase	Outcome	Gate
M0	External contracts and development runtime verified	No unknown critical dependency
M1	Working local skeleton	Clean build + tests
M2	Safe GW2 data gateway	Cached, throttled, fixture-tested data
M3	Deterministic market engine	Correct copper/fees/depth/scoring
M4	Useful desktop MVP	User can find and inspect opportunities
M5	Account/crafting mode	Feasible and economic crafting analysis
M6	Personal history	Realized profit is reproducible
M7	Historical research	Local evidence without prediction claims
M8	Release-ready local tool	Security, UX, performance, reproducible setup

Do not skip M0. It contains the model/runtime and API validation gates.

11 ADR-001 - Application Stack

11.1 Status

Accepted

11.2 Decision

Use ASP.NET Core/.NET 10 for the backend, React + TypeScript for the frontend, SQLite for local persistence, xUnit for .NET tests, and Playwright for browser-level tests.

11.3 Rationale

Clear separation of deterministic business logic from browser UI; strong .NET tooling; mature SQLite support; good fit for local desktop use and future hosted deployment.

11.4 Alternative

Blazor Web App remains a valid alternative if keeping one language becomes more important than the current frontend separation.

12 ADR-002 - Local-First Deployment

12.1 Status

Accepted

12.2 Decision

V1 runs entirely on the user's Mac with a loopback server and SQLite. No cloud server is required.

12.3 Consequences

Simpler security and privacy model; backups are the user's responsibility; future multi-user deployment will require a separate architecture review.

13 ADR-003 - No Application LLM in Current Scope

13.1 Status

Accepted

13.2 Decision

The application contains no LLM integration in the initial architecture. Qwen3.8-27B via MTPLX is a development agent only.

13.3 Rationale

Financial calculations and API semantics must remain deterministic, testable, reproducible, and independent of model behavior.

13.4 Future

A future LLM may explain structured results through an adapter without accessing secrets or executing actions.

14 ADR-004 - Single GW2 API Gateway

14.1 Status

Accepted

14.2 Decision

All Guild Wars 2 API requests go through one typed gateway abstraction with caching, batching, deduplication, scheduling, retries, and response validation.

14.3 Rationale

Prevents request multiplication, centralizes rate-limit policy, and isolates external schema changes.

15 ADR-005 - Integer Copper Money

15.1 Status

Accepted

15.2 Decision

Represent money as integer copper.

15.3 Rationale

Avoid floating-point rounding errors and make financial tests exact.

16 ADR-006 - Local Secret Storage

16.1 Status

Accepted

16.2 Decision

API credentials are stored outside source code using an OS-backed secret mechanism.

16.3 Development fallback

An environment variable may be used for local development and test execution only; it must never be committed.

17 ADR-007 - Read-Only External Boundary

17.1 Status

Accepted

17.2 Decision

The GW2 adapter exposes only explicitly supported GET resources. There is no generic authenticated request method available to feature code.

17.3 Rationale

Makes the project's read-only promise structural and testable.

18 Milestone and Ticket Index

Milestone	Name	Tickets
M0	Discovery and external-contract validation	TKT-M0-01, TKT-M0-02, TKT-M0-03, TKT-M0-04
M1	Repository and development foundation	TKT-M1-01, TKT-M1-02, TKT-M1-03, TKT-M1-04
M2	GW2 data gateway and caching	TKT-M2-01, TKT-M2-02, TKT-M2-03, TKT-M2-04

Milestone	Name	Tickets
M3	Deterministic market engine	TKT-M3-01, TKT-M3-02, TKT-M3-03, TKT-M3-04
M4	Dashboard and session planning	TKT-M4-01, TKT-M4-02, TKT-M4-03, TKT-M4-04
M5	Account-aware analysis and crafting	TKT-M5-01, TKT-M5-02, TKT-M5-03, TKT-M5-04
M6	Personal history and reconciliation	TKT-M6-01, TKT-M6-02, TKT-M6-03, TKT-M6-04
M7	Historical market data and investment research	TKT-M7-01, TKT-M7-02, TKT-M7-03, TKT-M7-04
M8	Hardening, accessibility, release readiness	TKT-M8-01, TKT-M8-02, TKT-M8-03, TKT-M8-04

19 M0 - Discovery and external-contract validation

19.1 Goal

Validate the model/runtime path, GW2 API capabilities, permissions, fees, quotas, and project constraints before implementation.

19.2 Dependencies

See the roadmap in `docs/specs/project-spec.md` and ticket dependency fields.

19.3 Completion rule

All tickets in this milestone meet acceptance criteria, tests pass, and any VERIFY items are either resolved or explicitly carried forward.

20 M1 - Repository and development foundation

20.1 Goal

Create the solution, modules, tests, CI-friendly commands, local configuration, and secure local runtime skeleton.

20.2 Dependencies

See the roadmap in `docs/specs/project-spec.md` and ticket dependency fields.

20.3 Completion rule

All tickets in this milestone meet acceptance criteria, tests pass, and any VERIFY items are either resolved or explicitly carried forward.

21 M2 - GW2 data gateway and caching

21.1 Goal

Build the typed read-only API client, request scheduling, caching, batching, fixtures, and observability.

21.2 Dependencies

See the roadmap in `docs/specs/project-spec.md` and ticket dependency fields.

21.3 Completion rule

All tickets in this milestone meet acceptance criteria, tests pass, and any VERIFY items are either resolved or explicitly carried forward.

22 M3 - Deterministic market engine

22.1 Goal

Implement money, fee policy, top-of-book screening, order-book simulation, liquidity, freshness, and opportunity scoring.

22.2 Dependencies

See the roadmap in `docs/specs/project-spec.md` and ticket dependency fields.

22.3 Completion rule

All tickets in this milestone meet acceptance criteria, tests pass, and any VERIFY items are either resolved or explicitly carried forward.

23 M4 - Dashboard and session planning

23.1 Goal

Deliver the first genuinely useful desktop UI and session planner around market opportunities.

23.2 Dependencies

See the roadmap in `docs/specs/project-spec.md` and ticket dependency fields.

23.3 Completion rule

All tickets in this milestone meet acceptance criteria, tests pass, and any VERIFY items are either resolved or explicitly carried forward.

24 M5 - Account-aware analysis and crafting

24.1 Goal

Add API-key validation, account snapshots, owned-item economics, crafting feasibility, and bounded multi-step crafting analysis.

24.2 Dependencies

See the roadmap in `docs/specs/project-spec.md` and ticket dependency fields.

24.3 Completion rule

All tickets in this milestone meet acceptance criteria, tests pass, and any VERIFY items are either resolved or explicitly carried forward.

25 M6 - Personal history and reconciliation

25.1 Goal

Persist saved operations and realized outcomes; distinguish realized and unrealized profit; support local clearing.

25.2 Dependencies

See the roadmap in docs/specs/project-spec.md and ticket dependency fields.

25.3 Completion rule

All tickets in this milestone meet acceptance criteria, tests pass, and any VERIFY items are either resolved or explicitly carried forward.

26 M7 - Historical market data and investment research

26.1 Goal

Collect local market snapshots efficiently and provide descriptive long-term analysis without pretending to forecast the future.

26.2 Dependencies

See the roadmap in docs/specs/project-spec.md and ticket dependency fields.

26.3 Completion rule

All tickets in this milestone meet acceptance criteria, tests pass, and any VERIFY items are either resolved or explicitly carried forward.

27 M8 - Hardening, accessibility, release readiness

27.1 Goal

Security review, browser compatibility, performance checks, documentation, backups, and a clean local release workflow.

27.2 Dependencies

See the roadmap in docs/specs/project-spec.md and ticket dependency fields.

27.3 Completion rule

All tickets in this milestone meet acceptance criteria, tests pass, and any VERIFY items are either resolved or explicitly carried forward.

28 TKT-M0-01 - Verify MTPLX/Qwen3.8-27B development model path

28.1 Milestone

M0

28.2 Goal

Determine whether the user-provided Qwen3.8-27B-GGUF can be used directly, must be converted, or should be replaced by an MTPLX-compatible Qwen3.8 27B artifact.

28.3 Dependencies

None

28.4 Acceptance criteria

- ☐ Inspect current MTPLX documentation and model compatibility behavior.
- ☐ Run the smallest safe model inspection/load command available on the Mac.
- ☐ Document whether raw GGUF loads directly.
- ☐ Document the chosen MTPLX-compatible artifact and provenance if conversion/catalog artifact is required.
- ☐ Do not modify application code to couple it to MTPLX.

28.5 Required tests

- ☐ A reproducible smoke command succeeds or fails with an explicit documented reason.
- ☐ The decision is recorded in the ticket/ADR notes.

28.6 Non-goals

- Changing the application architecture.
- Training a model.

28.7 Specification references

- docs/specs/project-spec.md
- docs/architecture/architecture.md
- docs/testing/testing-strategy.md
- Relevant milestone context file

28.8 Agent prompt

See the dedicated prompt file: docs/prompts/{code}-prompt.md.

29 TKT-M0-02 - Validate GW2 API endpoint and permission matrix

29.1 Milestone

M0

29.2 Goal

Create the authoritative endpoint table used by the application.

29.3 Dependencies

None

29.4 Acceptance criteria

- ☐ List every MVP endpoint, purpose, required permission, batching capability, expected freshness, and cache policy.
- ☐ Mark every uncertain fact VERIFY.
- ☐ Include prices, listings, recipes/search, account tokeninfo, bank/materials, character crafting/recipes, and any other endpoint actually required.
- ☐ No undocumented endpoint is added as a dependency.

29.5 Required tests

- ☐ Review table against current documentation URLs.
- ☐ At least one saved response fixture exists for each endpoint category later required by M2.

29.6 Non-goals

- Implementing the HTTP client.

29.7 Specification references

- docs/specs/project-spec.md
- docs/architecture/architecture.md
- docs/testing/testing-strategy.md
- Relevant milestone context file

29.8 Agent prompt

See the dedicated prompt file: docs/prompts/{code}-prompt.md.

30 TKT-M0-03 - Validate API quotas, caching guidance, and error behavior

30.1 Milestone

M0

30.2 Goal

Turn rate-limit and external-contract assumptions into measurable configuration requirements.

30.3 Dependencies

None

30.4 Acceptance criteria

- ☐ Document current rate-limit guidance from authoritative/community documentation.
- ☐ Define application-level configurable scheduler parameters without hard-coding an unverified quota.
- ☐ Document 429 handling and retry behavior.
- ☐ Define which live verification is safe and how it will be performed without deliberately stressing the API.

30.5 Required tests

- ☐ A rate-limit policy document exists.
- ☐ A mock 429 scenario is specified.

30.6 Non-goals

- Load testing the real GW2 API.

30.7 Specification references

- docs/specs/project-spec.md
- docs/architecture/architecture.md
- docs/testing/testing-strategy.md
- Relevant milestone context file

30.8 Agent prompt

See the dedicated prompt file: docs/prompts/{code}-prompt.md.

31 TKT-M0-04 - Validate legal/security scope and read-only boundary

31.1 Milestone

M0

31.2 Goal

Produce the release gate for terms, API-key handling, local security, and the strict read-only boundary.

31.3 Dependencies

None

31.4 Acceptance criteria

- ☐ Document current API terms references.
- ☐ Document personal-use GDPR/French security assumptions and what changes for public deployment.
- ☐ Define minimum API-key permissions by feature.
- ☐ Define explicit prohibited operations and a testable architecture rule preventing generic writes.

31.5 Required tests

- ☐ Security checklist exists.
- ☐ Read-only regression test idea is recorded.

31.6 Non-goals

- Legal advice beyond documenting requirements and assumptions.

31.7 Specification references

- docs/specs/project-spec.md
- docs/architecture/architecture.md
- docs/testing/testing-strategy.md
- Relevant milestone context file

31.8 Agent prompt

See the dedicated prompt file: `docs/prompts/{code}-prompt.md`.

32 TKT-M1-01 - Initialize repository and solution skeleton

32.1 Milestone

M1

32.2 Goal

Create the .NET/React repository structure and baseline developer tooling.

32.3 Dependencies

M0-04

32.4 Acceptance criteria

- ☐ Create src and tests structure from architecture.md.
- ☐ Create solution/project files with nullable and warnings-as-errors enabled where feasible.
- ☐ Create a minimal React frontend and backend that can be started locally.
- ☐ Add root README development commands.
- ☐ No business logic yet.

32.5 Required tests

- ☐ Clean checkout builds backend.
- ☐ Frontend installs/builds.
- ☐ Smoke test confirms local app starts.

32.6 Non-goals

- Implementing GW2 API access.

32.7 Specification references

- `docs/specs/project-spec.md`
- `docs/architecture/architecture.md`
- `docs/testing/testing-strategy.md`
- Relevant milestone context file

32.8 Agent prompt

See the dedicated prompt file: `docs/prompts/{code}-prompt.md`.

33 TKT-M1-02 - Add local configuration and secret-store abstraction

33.1 Milestone

M1

33.2 Goal

Create configuration interfaces that never require putting the API key in source control.

33.3 Dependencies

M1-01

33.4 Acceptance criteria

- ☐ Add an abstraction for secret retrieval/storage.
- ☐ Support a local development environment variable override.
- ☐ Ensure API key values are excluded from logs and browser responses.
- ☐ Add clear configuration error messages when a secret is missing.

33.5 Required tests

- ☐ Missing-secret test.
- ☐ Log redaction test.
- ☐ Browser endpoint does not expose secret test.

33.6 Non-goals

- Adding cloud secret management.

33.7 Specification references

- docs/specs/project-spec.md
- docs/architecture/architecture.md
- docs/testing/testing-strategy.md
- Relevant milestone context file

33.8 Agent prompt

See the dedicated prompt file: docs/prompts/{code}-prompt.md.

34 TKT-M1-03 - Add test infrastructure and fixture conventions

34.1 Milestone

M1

34.2 Goal

Create reusable test helpers and a fixture policy.

34.3 Dependencies

M1-01

34.4 Acceptance criteria

- ☐ Create unit, integration, and browser-test projects or equivalent harnesses.
- ☐ Create fixture folders and naming conventions.
- ☐ Create deterministic clock/test data helpers where needed.
- ☐ Document test commands.

34.5 Required tests

- ☐ All test projects execute with zero tests failing.
- ☐ One representative test exists at each layer.

34.6 Non-goals

- Real API calls in normal test execution.

34.7 Specification references

- docs/specs/project-spec.md
- docs/architecture/architecture.md
- docs/testing/testing-strategy.md
- Relevant milestone context file

34.8 Agent prompt

See the dedicated prompt file: docs/prompts/{code}-prompt.md.

35 TKT-M1-04 - Add local-only runtime and baseline security middle-ware

35.1 Milestone

M1

35.2 Goal

Make loopback-only operation and safe HTTP defaults explicit.

35.3 Dependencies

M1-01,M1-02

35.4 Acceptance criteria

- ☐ Default development server binds to loopback.
- ☐ Add safe response headers compatible with the local app.
- ☐ Validate inbound query/body values at API boundaries.
- ☐ Document how to intentionally change the binding and why doing so is not recommended.

35.5 Required tests

- ☐ Configuration test asserts loopback default.
- ☐ Security header smoke test.

35.6 Non-goals

- LAN/public hosting support.

35.7 Specification references

- docs/specs/project-spec.md
- docs/architecture/architecture.md
- docs/testing/testing-strategy.md
- Relevant milestone context file

35.8 Agent prompt

See the dedicated prompt file: docs/prompts/{code}-prompt.md.

36 TKT-M2-01 - Implement typed GW2 API client for public market data

36.1 Milestone

M2

36.2 Goal

Implement typed read-only requests for prices and listings based on verified schemas.

36.3 Dependencies

M0-02,M0-03,M1-03

36.4 Acceptance criteria

- ☐ Only explicit GET methods are exposed.
- ☐ External DTOs remain separate from domain models.
- ☐ Batch IDs are supported where the verified endpoint allows them.
- ☐ Unexpected fields do not crash parsing unnecessarily.
- ☐ Errors map to stable error categories.

36.5 Required tests

- ☐ Fixture tests for 200 responses.
- ☐ Malformed JSON test.
- ☐ HTTP error mapping tests.

36.6 Non-goals

- Account/authenticated endpoints other than those explicitly needed by later tickets.

36.7 Specification references

- docs/specs/project-spec.md
- docs/architecture/architecture.md
- docs/testing/testing-strategy.md
- Relevant milestone context file

36.8 Agent prompt

See the dedicated prompt file: docs/prompts/{code}-prompt.md.

37 TKT-M2-02 - Build request scheduler, deduplication, and retry policy

37.1 Milestone

M2

37.2 Goal

Centralize GW2 request rate management.

37.3 Dependencies

M2-01

37.4 Acceptance criteria

- ☐ Requests are scheduled through one component.
- ☐ Concurrent identical requests are deduplicated.
- ☐ 429 responses trigger bounded backoff.
- ☐ Retry rules distinguish transient from permanent failures.
- ☐ Configured quotas are not hard-coded from an unverified assumption.

37.5 Required tests

- ☐ Concurrency deduplication test.
- ☐ 429 backoff test.
- ☐ No retry for permanent auth/permission failure.

37.6 Non-goals

- Stress-testing live GW2.

37.7 Specification references

- docs/specs/project-spec.md
- docs/architecture/architecture.md
- docs/testing/testing-strategy.md
- Relevant milestone context file

37.8 Agent prompt

See the dedicated prompt file: docs/prompts/{code}-prompt.md.

38 TKT-M2-03 - Implement cache with freshness metadata

38.1 Milestone

M2

38.2 Goal

Cache public market responses and expose data age.

38.3 Dependencies

M2-01,M2-02

38.4 Acceptance criteria

- ☐ Cache entries have capture time and expiry policy.
- ☐ Cache hit avoids network request.
- ☐ Cache invalidation/refresh is deterministic.
- ☐ Data freshness is available to analytics and UI.

38.5 Required tests

- ☐ Hit/miss tests.
- ☐ Expiry test.
- ☐ Concurrent cache fill test.

38.6 Non-goals

- Historical market database.

38.7 Specification references

- docs/specs/project-spec.md
- docs/architecture/architecture.md
- docs/testing/testing-strategy.md
- Relevant milestone context file

38.8 Agent prompt

See the dedicated prompt file: docs/prompts/{code}-prompt.md.

39 TKT-M2-04 - Add market-data observability and diagnostics

39.1 Milestone

M2

39.2 Goal

Make API use and cache behavior auditable.

39.3 Dependencies

M2-02,M2-03

39.4 Acceptance criteria

- ☐ Track request counts, cache hits/misses, latency, 429s, and parsing failures.
- ☐ Never log API keys or authorization headers.
- ☐ Provide a local diagnostic view or structured diagnostic endpoint that is safe to display.

39.5 Required tests

- ☐ Sensitive-value redaction test.
- ☐ Counter increments on mock requests.

39.6 Non-goals

- Remote telemetry.

39.7 Specification references

- docs/specs/project-spec.md
- docs/architecture/architecture.md
- docs/testing/testing-strategy.md
- Relevant milestone context file

39.8 Agent prompt

See the dedicated prompt file: `docs/prompts/{code}-prompt.md`.

40 TKT-M3-01 - Implement Money and fee policy

40.1 Milestone

M3

40.2 Goal

Create exact copper arithmetic and centralized transaction-fee policy.

40.3 Dependencies

M2-01

40.4 Acceptance criteria

- ☐ Money is represented in integer copper.
- ☐ No floating-point arithmetic is used for money calculations.
- ☐ Fee policy is isolated and configurable.
- ☐ Profit formulas document scenario semantics.

40.5 Required tests

- ☐ Representative fee/profit examples.
- ☐ Boundary/large-value tests.
- ☐ Rounding tests.

40.6 Non-goals

- Hard-coding unexplained fee constants throughout the application.

40.7 Specification references

- `docs/specs/project-spec.md`
- `docs/architecture/architecture.md`
- `docs/testing/testing-strategy.md`
- Relevant milestone context file

40.8 Agent prompt

See the dedicated prompt file: `docs/prompts/{code}-prompt.md`.

41 TKT-M3-02 - Implement order-book quantity simulation

41.1 Milestone

M3

41.2 Goal

Calculate acquisition and liquidation scenarios across order-book levels.

41.3 Dependencies

M2-01,M3-01

41.4 Acceptance criteria

- ☐ Simulate quantity across multiple buy/sell levels.
- ☐ Calculate weighted average execution price and price impact.
- ☐ Handle insufficient depth explicitly.
- ☐ Return transparent assumptions and remaining quantity.

41.5 Required tests

- ☐ Single-level.
- ☐ Multiple-level.
- ☐ Partial-depth.
- ☐ Empty-book.
- ☐ Large quantity.

41.6 Non-goals

- Claiming guaranteed real-world fill behavior.

41.7 Specification references

- docs/specs/project-spec.md
- docs/architecture/architecture.md
- docs/testing/testing-strategy.md
- Relevant milestone context file

41.8 Agent prompt

See the dedicated prompt file: docs/prompts/{code}-prompt.md.

42 TKT-M3-03 - Implement flip profitability and liquidity analysis

42.1 Milestone

M3

42.2 Goal

Generate deterministic flip scenarios and a liquidity proxy.

42.3 Dependencies

M3-01,M3-02

42.4 Acceptance criteria

- ☐ Compute modeled net profit, ROI, capital required, price impact, and liquidity metrics.
- ☐ Support configurable minimum profit and capital constraints.
- ☐ Mark missing/stale data as unusable or lower confidence according to policy.
- ☐ Each result explains the scenario used.

42.5 Required tests

- ☐ Known-good fixture.
- ☐ Known-bad negative-profit fixture.
- ☐ Stale-data fixture.
- ☐ Insufficient-depth fixture.

42.6 Non-goals

- Automated orders.

42.7 Specification references

- docs/specs/project-spec.md
- docs/architecture/architecture.md
- docs/testing/testing-strategy.md
- Relevant milestone context file

42.8 Agent prompt

See the dedicated prompt file: docs/prompts/{code}-prompt.md.

43 TKT-M3-04 - Implement deterministic opportunity scoring

43.1 Milestone

M3

43.2 Goal

Rank opportunities using transparent weights and penalties.

43.3 Dependencies

M3-03

43.4 Acceptance criteria

- ☐ Score is deterministic for identical inputs/configuration.
- ☐ Weights can be configured without changing code.
- ☐ Score includes at least profit, capital efficiency, liquidity, freshness, risk, and complexity where applicable.
- ☐ UI-facing explanation metadata is produced.

43.5 Required tests

- ☐ Ordering tests.
- ☐ Tie tests.
- ☐ Weight sensitivity tests.
- ☐ Stale-data penalty test.

43.6 Non-goals

- LLM-generated ranking.

43.7 Specification references

- docs/specs/project-spec.md
- docs/architecture/architecture.md
- docs/testing/testing-strategy.md
- Relevant milestone context file

43.8 Agent prompt

See the dedicated prompt file: docs/prompts/{code}-prompt.md.

44 TKT-M4-01 - Create market dashboard and filters

44.1 Milestone

M4

44.2 Goal

Build the main desktop dashboard around deterministic opportunities.

44.3 Dependencies

M3-04

44.4 Acceptance criteria

- ☐ Show ranked opportunities with capital, modeled profit, ROI, liquidity, risk/confidence, and data age.
- ☐ Provide filters for capital, minimum profit, strategy, risk, and freshness.
- ☐ No unsupported guarantee language.
- ☐ Empty/loading/error states are explicit.

44.5 Required tests

- ☐ Component tests for filters.
- ☐ Browser smoke test for dashboard.

44.6 Non-goals

- Mobile-first UI.

44.7 Specification references

- docs/specs/project-spec.md
- docs/architecture/architecture.md
- docs/testing/testing-strategy.md
- Relevant milestone context file

44.8 Agent prompt

See the dedicated prompt file: docs/prompts/{code}-prompt.md.

45 TKT-M4-02 - Create opportunity detail view

45.1 Milestone

M4

45.2 Goal

Expose calculation details and assumptions.

45.3 Dependencies

M4-01

45.4 Acceptance criteria

- ☐ Show acquisition/exit assumptions, fees, modeled profit, ROI, capital, order-book impact, liquidity, and data age.
- ☐ Provide a human-readable calculation breakdown.
- ☐ Clearly distinguish scenario values from actual outcomes.

45.5 Required tests

- ☐ Detail rendering test.
- ☐ Fixture comparison test.

45.6 Non-goals

- Executing actions from the detail page.

45.7 Specification references

- docs/specs/project-spec.md
- docs/architecture/architecture.md
- docs/testing/testing-strategy.md
- Relevant milestone context file

45.8 Agent prompt

See the dedicated prompt file: docs/prompts/{code}-prompt.md.

46 TKT-M4-03 - Implement user session preferences

46.1 Milestone

M4

46.2 Goal

Persist user capital and opportunity preferences locally.

46.3 Dependencies

M4-01

46.4 Acceptance criteria

- ☐ Store capital, minimum profit, risk preference, strategy preference, and allocation constraints locally.
- ☐ Validate numeric ranges and sensible defaults.
- ☐ Changing preferences re-ranks or filters results deterministically.

46.5 Required tests

- ☐ Persistence test.
- ☐ Validation test.
- ☐ Preference-to-score/filter test.

46.6 Non-goals

- Remote user accounts.

46.7 Specification references

- docs/specs/project-spec.md
- docs/architecture/architecture.md
- docs/testing/testing-strategy.md
- Relevant milestone context file

46.8 Agent prompt

See the dedicated prompt file: docs/prompts/{code}-prompt.md.

47 TKT-M4-04 - Implement session planner with coarse effort categories

47.1 Milestone

M4

47.2 Goal

Turn constraints into a practical shortlist without pretending to know execution time.

47.3 Dependencies

M4-03,M3-04

47.4 Acceptance criteria

- ☐ Support effort categories: very low, low, medium, high, ongoing/patient.
- ☐ Planner respects capital/risk/preferences.
- ☐ Planner produces an ordered session shortlist.
- ☐ UI states that categories are approximations, not time guarantees.

47.5 Required tests

- ☐ Constraint tests.
- ☐ Insufficient-capital test.
- ☐ Risk-filter test.

47.6 Non-goals

- Precise execution-time prediction.

47.7 Specification references

- docs/specs/project-spec.md
- docs/architecture/architecture.md
- docs/testing/testing-strategy.md
- Relevant milestone context file

47.8 Agent prompt

See the dedicated prompt file: docs/prompts/{code}-prompt.md.

48 TKT-M5-01 - Implement token validation and permission-aware account access

48.1 Milestone

M5

48.2 Goal

Validate an API key and expose only permitted features.

48.3 Dependencies

M0-02,M0-04,M1-02,M2-02

48.4 Acceptance criteria

- ☐ Use tokeninfo or the verified equivalent to validate the key.
- ☐ Display safe permission status.
- ☐ Never return the key to the browser.
- ☐ Disable unsupported account features when scopes are missing.
- ☐ Render token metadata such as key name as text, not HTML.

48.5 Required tests

- ☐ Valid token fixture.
- ☐ Missing permission fixture.
- ☐ Malformed metadata/XSS string fixture.
- ☐ Secret non-disclosure test.

48.6 Non-goals

- Storing the API key in frontend localStorage.

48.7 Specification references

- docs/specs/project-spec.md
- docs/architecture/architecture.md
- docs/testing/testing-strategy.md
- Relevant milestone context file

48.8 Agent prompt

See the dedicated prompt file: docs/prompts/{code}-prompt.md.

49 TKT-M5-02 - Implement account snapshot ingestion

49.1 Milestone

M5

49.2 Goal

Add bank/materials and relevant character data with caching and local scoping.

49.3 Dependencies

M5-01,M2-03

49.4 Acceptance criteria

- ☐ Fetch only endpoints required for enabled account features.
- ☐ Cache account snapshots separately from public market data.
- ☐ Associate snapshots with a local account profile identifier.
- ☐ Handle missing permissions and partial data gracefully.

49.5 Required tests

- ☐ Fixture ingestion tests.
- ☐ Permission failure test.
- ☐ Cache isolation test.

49.6 Non-goals

- Persisting unnecessary full raw payloads indefinitely.

49.7 Specification references

- docs/specs/project-spec.md
- docs/architecture/architecture.md
- docs/testing/testing-strategy.md
- Relevant milestone context file

49.8 Agent prompt

See the dedicated prompt file: docs/prompts/{code}-prompt.md.

50 TKT-M5-03 - Implement owned-item opportunity cost

50.1 Milestone

M5

50.2 Goal

Value owned materials economically and compare with buying them.

50.3 Dependencies

M3-01,M5-02

50.4 Acceptance criteria

- ☐ Owned materials are never assumed free.
- ☐ Compute a realizable economic value using configured market evidence.
- ☐ Support owned/buy/mixed strategies where data is sufficient.
- ☐ Flag bound/unavailable/non-sellable items where relevant data exists.

50.5 Required tests

- ☐ Owned vs bought example.
- ☐ Partial owned stock.
- ☐ Unavailable price.
- ☐ Non-economic ownership restriction.

50.6 Non-goals

- Assuming all inventory is sellable merely because it has an item ID.

50.7 Specification references

- docs/specs/project-spec.md
- docs/architecture/architecture.md
- docs/testing/testing-strategy.md
- Relevant milestone context file

50.8 Agent prompt

See the dedicated prompt file: docs/prompts/{code}-prompt.md.

51 TKT-M5-04 - Implement bounded crafting path search

51.1 Milestone

M5

51.2 Goal

Analyze profitable crafting paths with feasibility constraints and controlled recursion.

51.3 Dependencies

M5-02,M5-03,M3-03

51.4 Acceptance criteria

- ☐ Recipe graph represents output/ingredient relationships.
- ☐ Apply discipline/rating and recipe availability constraints when verified data exists.
- ☐ Search detects cycles and has configurable depth/candidate limits.
- ☐ Memoization prevents repeated subproblem explosion.
- ☐ Analysis reports truncation/unknowns explicitly.
- ☐ Compare purchase cost versus owned-material opportunity cost.

51.5 Required tests

- ☐ Simple recipe.
- ☐ Multi-step recipe.
- ☐ Cycle.

- ☐ Depth cap.
- ☐ Alternative path.
- ☐ Mixed owned/purchased ingredients.

51.6 Non-goals

- Exhaustive unlimited world optimization.

51.7 Specification references

- docs/specs/project-spec.md
- docs/architecture/architecture.md
- docs/testing/testing-strategy.md
- Relevant milestone context file

51.8 Agent prompt

See the dedicated prompt file: docs/prompts/{code}-prompt.md.

52 TKT-M6-01 - Create operation history model and local persistence

52.1 Milestone

M6

52.2 Goal

Store saved/planned operations and their calculation context.

52.3 Dependencies

M4-04

52.4 Acceptance criteria

- ☐ Store operations with timestamps and calculation/configuration version identifiers.
- ☐ Preserve the scenario assumptions used at save time.
- ☐ Allow user to mark status such as planned/in-progress/completed/cancelled.

52.5 Required tests

- ☐ Persistence round trip.
- ☐ Schema migration test.

52.6 Non-goals

- Syncing history to a cloud service.

52.7 Specification references

- docs/specs/project-spec.md
- docs/architecture/architecture.md
- docs/testing/testing-strategy.md
- Relevant milestone context file

52.8 Agent prompt

See the dedicated prompt file: `docs/prompts/{code}-prompt.md`.

53 TKT-M6-02 - Implement realized profit reconciliation

53.1 Milestone

M6

53.2 Goal

Calculate realized profit separately from unrealized P/L.

53.3 Dependencies

M6-01,M3-01

53.4 Acceptance criteria

- ☐ Realized profit uses recorded actual acquisition/sale values and fees.
- ☐ Unrealized value is separately labeled.
- ☐ Incomplete operations are not silently counted as realized profit.
- ☐ Profit statistics are reproducible from stored records.

53.5 Required tests

- ☐ Complete trade.
- ☐ Partial trade.
- ☐ Cancelled trade.
- ☐ Fee variation.
- ☐ Unrealized-only record.

53.6 Non-goals

- Claiming lifetime profit before local history starts.

53.7 Specification references

- `docs/specs/project-spec.md`
- `docs/architecture/architecture.md`
- `docs/testing/testing-strategy.md`
- Relevant milestone context file

53.8 Agent prompt

See the dedicated prompt file: `docs/prompts/{code}-prompt.md`.

54 TKT-M6-03 - Build personal dashboard/history statistics

54.1 Milestone

M6

54.2 Goal

Show useful local statistics without overstating their meaning.

54.3 Dependencies

M6-02

54.4 Acceptance criteria

- ☐ Show operation count, realized profit since first recorded use, average modeled/realized metrics where justified, and completion rate where defined.
- ☐ Clearly show the historical coverage period.
- ☐ Provide empty-state messaging for insufficient history.

54.5 Required tests

- ☐ Statistics fixtures.
- ☐ Empty-history test.

54.6 Non-goals

- Backfilling unknown lifetime history.

54.7 Specification references

- docs/specs/project-spec.md
- docs/architecture/architecture.md
- docs/testing/testing-strategy.md
- Relevant milestone context file

54.8 Agent prompt

See the dedicated prompt file: docs/prompts/{code}-prompt.md.

55 TKT-M6-04 - Implement local data clearing and backup guidance

55.1 Milestone

M6

55.2 Goal

Give the user control over local account/history data.

55.3 Dependencies

M6-01,M5-02

55.4 Acceptance criteria

- ☐ Provide a user-visible clear account data action.
- ☐ Distinguish account snapshots from public cache/history where sensible.
- ☐ Document SQLite backup/restore at a user level.
- ☐ No data is silently uploaded.

55.5 Required tests

- ☐ Clear-data integration test.
- ☐ Backup documentation review.

55.6 Non-goals

- Automated cloud backup.

55.7 Specification references

- docs/specs/project-spec.md
- docs/architecture/architecture.md
- docs/testing/testing-strategy.md
- Relevant milestone context file

55.8 Agent prompt

See the dedicated prompt file: docs/prompts/{code}-prompt.md.

56 TKT-M7-01 - Design local market snapshot schema and sampling policy

56.1 Milestone

M7

56.2 Goal

Create the foundation for historical data without over-collecting.

56.3 Dependencies

M2-03,M6-04

56.4 Acceptance criteria

- ☐ Store timestamped price/order-book snapshots with source freshness.
- ☐ Define sampling policy by item class/watchlist.
- ☐ Estimate local storage growth.
- ☐ Allow future changes without corrupting existing history.

56.5 Required tests

- ☐ Schema migration test.
- ☐ Sampling policy unit tests.

56.6 Non-goals

- Full-frequency capture of every item.

56.7 Specification references

- docs/specs/project-spec.md
- docs/architecture/architecture.md
- docs/testing/testing-strategy.md
- Relevant milestone context file

56.8 Agent prompt

See the dedicated prompt file: `docs/prompts/{code}-prompt.md`.

57 TKT-M7-02 - Implement economical historical collection scheduler

57.1 Milestone

M7

57.2 Goal

Collect snapshots while minimizing API requests.

57.3 Dependencies

M7-01,M2-02

57.4 Acceptance criteria

- ☐ High-interest/watchlist items can be sampled more frequently.
- ☐ Low-interest items use a lower sampling rate.
- ☐ Scheduler respects API request budget.
- ☐ Collection pauses cleanly on rate limiting or application shutdown.

57.5 Required tests

- ☐ Schedule calculation tests.
- ☐ Request budget tests.
- ☐ Pause/resume tests.

57.6 Non-goals

- Aggressive historical scraping.

57.7 Specification references

- `docs/specs/project-spec.md`
- `docs/architecture/architecture.md`
- `docs/testing/testing-strategy.md`
- Relevant milestone context file

57.8 Agent prompt

See the dedicated prompt file: `docs/prompts/{code}-prompt.md`.

58 TKT-M7-03 - Implement descriptive historical analytics

58.1 Milestone

M7

58.2 Goal

Compute percentiles, volatility, drawdown, spread persistence, and liquidity stability from local observations.

58.3 Dependencies

M7-01,M7-02

58.4 Acceptance criteria

- ☐ Historical metrics are computed only from available local observations.
- ☐ Sufficient sample checks prevent misleading statistics.
- ☐ Charts/metrics disclose observation window and sample count.
- ☐ No claim of future prediction is made.

58.5 Required tests

- ☐ Known time series metrics.
- ☐ Insufficient sample test.
- ☐ Missing observation test.

58.6 Non-goals

- Machine-learning price forecasts.

58.7 Specification references

- docs/specs/project-spec.md
- docs/architecture/architecture.md
- docs/testing/testing-strategy.md
- Relevant milestone context file

58.8 Agent prompt

See the dedicated prompt file: docs/prompts/{code}-prompt.md.

59 TKT-M7-04 - Create investment-research view with uncertainty

59.1 Milestone

M7

59.2 Goal

Present long-term observations as evidence, not certainty.

59.3 Dependencies

M7-03,M4-02

59.4 Acceptance criteria

- ☐ Show historical range/percentile and liquidity context.
- ☐ Show data coverage and sample count.
- ☐ Use language such as observed/estimated rather than guaranteed.
- ☐ Allow the user to compare a small watchlist.

59.5 Required tests

- ☐ Coverage disclosure test.
- ☐ Watchlist comparison test.

59.6 Non-goals

- Automated investment advice presented as fact.

59.7 Specification references

- docs/specs/project-spec.md
- docs/architecture/architecture.md
- docs/testing/testing-strategy.md
- Relevant milestone context file

59.8 Agent prompt

See the dedicated prompt file: docs/prompts/{code}-prompt.md.

60 TKT-M8-01 - Perform security review and secret-leak audit

60.1 Milestone

M8

60.2 Goal

Verify no API key or sensitive account data leaks through code, logs, browser, fixtures, or Git.

60.3 Dependencies

M6-04,M5-01

60.4 Acceptance criteria

- ☐ Run secret scanning on repository.
- ☐ Inspect network/browser responses for API-key disclosure.
- ☐ Inspect logs and exception pages for secrets.
- ☐ Confirm local bind/security defaults.
- ☐ Document remaining risks.

60.5 Required tests

- ☐ Automated secret-pattern test where appropriate.
- ☐ Manual security checklist sign-off.

60.6 Non-goals

- Guaranteeing zero vulnerabilities.

60.7 Specification references

- docs/specs/project-spec.md
- docs/architecture/architecture.md
- docs/testing/testing-strategy.md
- Relevant milestone context file

60.8 Agent prompt

See the dedicated prompt file: docs/prompts/{code}-prompt.md.

61 TKT-M8-02 - Perform browser/accessibility and UX hardening

61.1 Milestone

M8

61.2 Goal

Validate the desktop UI across target browsers and accessibility basics.

61.3 Dependencies

M4-04,M5-04,M6-03

61.4 Acceptance criteria

- ☐ Keyboard navigation works for primary flows.
- ☐ Focus states are visible.
- ☐ Critical information meets the chosen contrast standard.
- ☐ Loading/error/empty states are understandable.
- ☐ Run smoke tests against the documented browser matrix.

61.5 Required tests

- ☐ Playwright smoke suite.
- ☐ Accessibility scan appropriate to project toolchain.

61.6 Non-goals

- Mobile-first redesign.

61.7 Specification references

- docs/specs/project-spec.md
- docs/architecture/architecture.md
- docs/testing/testing-strategy.md
- Relevant milestone context file

61.8 Agent prompt

See the dedicated prompt file: docs/prompts/{code}-prompt.md.

62 TKT-M8-03 - Performance and API-request efficiency review

62.1 Milestone

M8

62.2 Goal

Ensure the MVP remains responsive and API-efficient.

62.3 Dependencies

M3-04,M5-04,M7-02

62.4 Acceptance criteria

- ☐ Benchmark representative analytical workloads with local fixtures.
- ☐ Confirm candidate screening prevents unnecessary deep listings requests.
- ☐ Confirm cache and request deduplication behavior.
- ☐ Document any known computational limits for crafting search.

62.5 Required tests

- ☐ Performance smoke benchmark.
- ☐ Request-count regression test.

62.6 Non-goals

- Optimizing prematurely at the cost of correctness.

62.7 Specification references

- docs/specs/project-spec.md
- docs/architecture/architecture.md
- docs/testing/testing-strategy.md
- Relevant milestone context file

62.8 Agent prompt

See the dedicated prompt file: docs/prompts/{code}-prompt.md.

63 TKT-M8-04 - Prepare local release package and handoff documentation

63.1 Milestone

M8

63.2 Goal

Make the project reproducible for the owner.

63.3 Dependencies

M8-01,M8-02,M8-03

63.4 Acceptance criteria

- ☐ One documented setup path works on a clean Mac environment.
- ☐ One documented run/test command set exists.
- ☐ Configuration and backup instructions are complete.
- ☐ Known limitations and deferred features are listed.
- ☐ No secrets or private data are included in the release package.

63.5 Required tests

- ☐ Clean-environment checklist.
- ☐ Release smoke test.

63.6 Non-goals

- Public deployment.

63.7 Specification references

- docs/specs/project-spec.md
- docs/architecture/architecture.md
- docs/testing/testing-strategy.md
- Relevant milestone context file

63.8 Agent prompt

See the dedicated prompt file: docs/prompts/{code}-prompt.md.

You are the implementation agent for the Tyrian Ledger project.

Read first: - docs/context/permanent-context.md - docs/context/milestone-context-M0.md - docs/tickets/TKT-M0-01.md

Your job is to implement ONLY this ticket. Do not redesign the application or add an LLM feature.

Rules: - Never invent GW2 API fields, permissions, quotas, or behavior. Mark uncertain facts VERIFY. - Preserve the read-only boundary. Do not add gameplay or Trading Post automation. - Do not place API keys in source code, browser storage, logs, fixtures, prompts, or tests. - Keep money calculations in integer copper. - Keep external API DTOs separate from domain models. - Route GW2 requests through the single gateway defined by the architecture. - Add or update tests for every behavior change. Never weaken or delete a test just to make it pass. - Minimize unrelated file changes.

Execution protocol: 1. Inspect the current repository and relevant existing code. 2. Restate the ticket acceptance criteria in implementation terms. 3. Identify any real contradiction or missing dependency before coding. If one exists, stop and explain it rather than inventing a solution. 4. Implement the smallest coherent change satisfying the ticket. 5. Add/update unit, integration, or browser tests as appropriate. 6. Run the narrow test set first, then the relevant broader test set. 7. Check formatting/analyzers/build. 8. Review the diff for accidental scope expansion and secret leakage. 9. Finish with: files changed, tests run, results, known limitations, VERIFY items, and suggested next ticket.

Ticket-specific objective: Determine whether the user-provided Qwen3.8-27B-GGUF can be used directly, must be converted, or should be replaced by an MTPLX-compatible Qwen3.8 27B artifact.

Ticket-specific acceptance criteria: - Inspect current MTPLX documentation and model compatibility behavior. - Run the smallest safe model inspection/load command available on the Mac. - Document whether raw GGUF loads directly. - Document the chosen MTPLX-compatible artifact and provenance if conversion/catalog artifact is required. - Do not modify application code to couple it to MTPLX.

Ticket-specific non-goals: - Changing the application architecture. - Training a model.

Delivery protocol (mandatory for every ticket): - Create/use a dedicated branch named `ticket/<TKT-M0-01>-<short-kebab-title>`. - Every commit for this ticket MUST start with `[TKT-M0-01]`. - Before declaring the ticket complete, push the branch and create a GitHub pull request titled `[TKT-M0-01] Short title`. - The PR body MUST identify the ticket and milestone and list the exact specification/architecture/ADR/testing/security/UX references implemented or validated, plus summary, acceptance-criteria status, validation, decisions, VERIFY items, risks/limitations, and follow-up. - Verify that the PR actually exists and report its URL. Never invent a PR URL. - Do not merge the PR. Human review and merge are required. - If GitHub CLI/authentication/permissions/remote access prevent PR creation, stop at the delivery gate and report the blocker; do not claim the ticket is complete.

You are the implementation agent for the Tyrian Ledger project.

Read first: - docs/context/permanent-context.md - docs/context/milestone-context-M0.md - docs/tickets/TKT-M0-02.md

Your job is to implement ONLY this ticket. Do not redesign the application or add an LLM feature.

Rules: - Never invent GW2 API fields, permissions, quotas, or behavior. Mark uncertain facts VERIFY. - Preserve the read-only boundary. Do not add gameplay or Trading Post automation. - Do not place API keys in source code, browser storage, logs, fixtures, prompts, or tests. - Keep money calculations in integer copper. - Keep external API DTOs separate from domain models. - Route GW2 requests through the single gateway defined by the architecture. - Add or update tests for every behavior change. Never weaken or delete a test just to make it pass. - Minimize unrelated file changes.

Execution protocol: 1. Inspect the current repository and relevant existing code. 2. Restate the ticket acceptance criteria in implementation terms. 3. Identify any real contradiction or missing dependency before coding. If one exists, stop and explain it rather than inventing a solution. 4. Implement the smallest coherent change satisfying the ticket. 5. Add/update unit, integration, or browser tests as appropriate. 6. Run the narrow test set first, then the relevant broader test set. 7. Check formatting/analyzers/build. 8. Review the diff for accidental scope expansion and secret leakage. 9. Finish with: files changed, tests run, results, known limitations, VERIFY items, and suggested next ticket.

Ticket-specific objective: Create the authoritative endpoint table used by the application.

Ticket-specific acceptance criteria: - List every MVP endpoint, purpose, required permission, batching capability, expected freshness, and cache policy. - Mark every uncertain fact VERIFY. - Include prices, listings, recipes/search, account tokeninfo, bank/materials, character crafting/recipes, and any other endpoint actually required. - No undocumented endpoint is added as a dependency.

Ticket-specific non-goals: - Implementing the HTTP client.

Delivery protocol (mandatory for every ticket): - Create/use a dedicated branch named `ticket/<TKT-M0-02>-<short-kebab-title>`. - Every commit for this ticket MUST start with `[TKT-M0-02]`. - Before declaring the ticket complete, push the branch and create a GitHub pull request titled `[TKT-M0-02] Short title`. - The PR body MUST identify the ticket and milestone and list the exact specification/architecture/ADR/testing/security/UX references implemented or validated, plus summary, acceptance-criteria status, validation, decisions, VERIFY items, risks/limitations, and follow-up. - Verify that the PR actually exists and report its URL. Never invent a PR URL. - Do not merge the PR. Human review and merge are required. - If GitHub CLI/authentication/permissions/remote access prevent PR creation, stop at the delivery gate and report the blocker; do not claim the ticket is complete.

You are the implementation agent for the Tyrian Ledger project.

Read first: - docs/context/permanent-context.md - docs/context/milestone-context-M0.md - docs/tickets/TKT-M0-03.md

Your job is to implement ONLY this ticket. Do not redesign the application or add an LLM feature.

Rules: - Never invent GW2 API fields, permissions, quotas, or behavior. Mark uncertain facts VERIFY. - Preserve the read-only boundary. Do not add gameplay or Trading Post automation. - Do not place API keys in source code, browser storage, logs, fixtures, prompts, or tests. - Keep money calculations in integer copper. - Keep external API DTOs separate from domain models. - Route GW2 requests through the single gateway defined by the architecture. - Add or update tests for every behavior change. Never weaken or delete a test just to make it pass. - Minimize unrelated file changes.

Execution protocol: 1. Inspect the current repository and relevant existing code. 2. Restate the ticket acceptance criteria in implementation terms. 3. Identify any real contradiction or missing dependency before coding. If one exists, stop and explain it rather than inventing a solution. 4. Implement the smallest coherent change satisfying the ticket. 5. Add/update unit, integration, or browser tests as appropriate. 6. Run the narrow test set first, then the relevant broader test set. 7. Check formatting/analyzers/build. 8. Review the diff for accidental scope expansion and secret leakage. 9. Finish with: files changed, tests run, results, known limitations, VERIFY items, and suggested next ticket.

Ticket-specific objective: Turn rate-limit and external-contract assumptions into measurable configuration requirements.

Ticket-specific acceptance criteria: - Document current rate-limit guidance from authoritative/community documentation. - Define application-level configurable scheduler parameters without hard-coding an unverified quota. - Document 429 handling and retry behavior. - Define which live verification is safe and how it will be performed without deliberately stressing the API.

Ticket-specific non-goals: - Load testing the real GW2 API.

Delivery protocol (mandatory for every ticket): - Create/use a dedicated branch named `ticket/<TKT-M0-03>-<short-kebab-title>`. - Every commit for this ticket MUST start with `[TKT-M0-03] Short title`. - The PR body MUST identify the ticket and milestone and list the exact specification/architecture/ADR/testing/security/UX references implemented or validated, plus summary, acceptance-criteria status, validation, decisions, VERIFY items, risks/limitations, and follow-up. - Verify that the PR actually exists and report its URL. Never invent a PR URL. - Do not merge the PR. Human review and merge are required. - If GitHub CLI/authentication/permissions/remote access prevent PR creation, stop at the delivery gate and report the blocker; do not claim the ticket is complete.

You are the implementation agent for the Tyrian Ledger project.

Read first: - docs/context/permanent-context.md - docs/context/milestone-context-M0.md - docs/tickets/TKT-M0-04.md

Your job is to implement ONLY this ticket. Do not redesign the application or add an LLM feature.

Rules: - Never invent GW2 API fields, permissions, quotas, or behavior. Mark uncertain facts VERIFY. - Preserve the read-only boundary. Do not add gameplay or Trading Post automation. - Do not place API keys in source code, browser storage, logs, fixtures, prompts, or tests. - Keep money calculations in integer copper. - Keep external API DTOs separate from domain models. - Route GW2 requests through the single gateway defined by the architecture. - Add or update tests for every behavior change. Never weaken or delete a test just to make it pass. - Minimize unrelated file changes.

Execution protocol: 1. Inspect the current repository and relevant existing code. 2. Restate the ticket acceptance criteria in implementation terms. 3. Identify any real contradiction or missing dependency before coding. If one exists, stop and explain it rather than inventing a solution. 4. Implement the smallest coherent change satisfying the ticket. 5. Add/update unit, integration, or browser tests as appropriate. 6. Run the narrow test set first, then the relevant broader test set. 7. Check formatting/analyzers/build. 8. Review the diff for accidental scope expansion and secret leakage. 9. Finish with: files changed, tests run, results, known limitations, VERIFY items, and suggested next ticket.

Ticket-specific objective: Produce the release gate for terms, API-key handling, local security, and the strict read-only boundary.

Ticket-specific acceptance criteria: - Document current API terms references. - Document personal-use GDPR/French security assumptions and what changes for public deployment. - Define minimum API-key permissions by feature. - Define explicit prohibited operations and a testable architecture rule preventing generic writes.

Ticket-specific non-goals: - Legal advice beyond documenting requirements and assumptions.

Delivery protocol (mandatory for every ticket): - Create/use a dedicated branch named `ticket/<TKT-M0-04>-<short-kebab-title>`. - Every commit for this ticket MUST start with `[TKT-M0-04] Short title`. - The PR body MUST identify the ticket and milestone and list the exact specification/architecture/ADR/testing/security/UX references implemented or validated, plus summary, acceptance-criteria status, validation, decisions, VERIFY items, risks/limitations, and follow-up. - Verify that the PR actually exists and report its URL. Never invent a PR URL. - Do not merge the PR. Human review and merge are required. - If GitHub CLI/authentication/permissions/remote access prevent PR creation, stop at the delivery gate and report the blocker; do not claim the ticket is complete.

You are the implementation agent for the Tyrian Ledger project.

Read first: - docs/context/permanent-context.md - docs/context/milestone-context-M1.md - docs/tickets/TKT-M1-01.md

Your job is to implement ONLY this ticket. Do not redesign the application or add an LLM feature.

Rules: - Never invent GW2 API fields, permissions, quotas, or behavior. Mark uncertain facts VERIFY. - Preserve the read-only boundary. Do not add gameplay or Trading Post automation. - Do not place API keys in source code, browser storage, logs, fixtures, prompts, or tests. - Keep money calculations in integer copper. - Keep external API DTOs separate from domain models. - Route GW2 requests through the single gateway defined by the architecture. - Add or update tests for every behavior change. Never weaken or delete a test just to make it pass. - Minimize unrelated file changes.

Execution protocol: 1. Inspect the current repository and relevant existing code. 2. Restate the ticket acceptance criteria in implementation terms. 3. Identify any real contradiction or missing dependency before coding. If one exists, stop and explain it rather than inventing a solution. 4. Implement the smallest coherent change satisfying the ticket. 5. Add/update unit, integration, or browser tests as appropriate. 6. Run the narrow test set first, then the relevant broader test set. 7. Check formatting/analyzers/build. 8. Review the diff for accidental scope expansion and secret leakage. 9. Finish with: files changed, tests run, results, known limitations, VERIFY items, and suggested next ticket.

Ticket-specific objective: Create the .NET/React repository structure and baseline developer tooling.

Ticket-specific acceptance criteria: - Create src and tests structure from architecture.md. - Create solution/project files with nullable and warnings-as-errors enabled where feasible. - Create a minimal React frontend and backend that can be started locally. - Add root README development commands. - No business logic yet.

Ticket-specific non-goals: - Implementing GW2 API access.

Delivery protocol (mandatory for every ticket): - Create/use a dedicated branch named `ticket/<TKT-M1-01>-<short-kebab-title>`. - Every commit for this ticket MUST start with `[TKT-M1-01]`. - Before declaring the ticket complete, push the branch and create a GitHub pull request titled `[TKT-M1-01] Short title`. - The PR body MUST identify the ticket and milestone and list the exact specification/architecture/ADR/testing/security/UX references implemented or validated, plus summary, acceptance-criteria status, validation, decisions, VERIFY items, risks/limitations, and follow-up. - Verify that the PR actually exists and report its URL. Never invent a PR URL. - Do not merge the PR. Human review and merge are required. - If GitHub CLI/authentication/permissions/remote access prevent PR creation, stop at the delivery gate and report the blocker; do not claim the ticket is complete.

You are the implementation agent for the Tyrian Ledger project.

Read first: - docs/context/permanent-context.md - docs/context/milestone-context-M1.md - docs/tickets/TKT-M1-02.md

Your job is to implement ONLY this ticket. Do not redesign the application or add an LLM feature.

Rules: - Never invent GW2 API fields, permissions, quotas, or behavior. Mark uncertain facts VERIFY. - Preserve the read-only boundary. Do not add gameplay or Trading Post automation. - Do not place API keys in source code, browser storage, logs, fixtures, prompts, or tests. - Keep money calculations in integer copper. - Keep external API DTOs separate from domain models. - Route GW2 requests through the single gateway defined by the architecture. - Add or update tests for every behavior change. Never weaken or delete a test just to make it pass. - Minimize unrelated file changes.

Execution protocol: 1. Inspect the current repository and relevant existing code. 2. Restate the ticket acceptance criteria in implementation terms. 3. Identify any real contradiction or missing dependency before coding. If one exists, stop and explain it rather than inventing a solution. 4. Implement the smallest coherent change satisfying the ticket. 5. Add/update unit, integration, or browser tests as appropriate. 6. Run the narrow test set first, then the relevant broader test set. 7. Check formatting/analyzers/build. 8. Review the diff for accidental scope expansion and secret leakage. 9. Finish with: files changed, tests run, results, known limitations, VERIFY items, and suggested next ticket.

Ticket-specific objective: Create configuration interfaces that never require putting the API key in source control.

Ticket-specific acceptance criteria: - Add an abstraction for secret retrieval/storage. - Support a local development environment variable override. - Ensure API key values are excluded from logs and browser responses. - Add clear configuration error messages when a secret is missing.

Ticket-specific non-goals: - Adding cloud secret management.

Delivery protocol (mandatory for every ticket): - Create/use a dedicated branch named `ticket/<TKT-M1-02>-<short-kebab-title>`. - Every commit for this ticket MUST start with `[TKT-M1-02]`. - Before declaring the ticket complete, push the branch and create a GitHub pull request titled `[TKT-M1-02] Short title`. - The PR body MUST identify the ticket and milestone and list the exact specification/architecture/ADR/testing/security/UX references implemented or validated, plus summary, acceptance-criteria status, validation, decisions, VERIFY items, risks/limitations, and follow-up. - Verify that the PR actually exists and report its URL. Never invent a PR URL. - Do not merge the PR. Human review and merge are required. - If GitHub CLI/authentication/permissions/remote access prevent PR creation, stop at the delivery gate and report the blocker; do not claim the ticket is complete.

You are the implementation agent for the Tyrian Ledger project.

Read first: - docs/context/permanent-context.md - docs/context/milestone-context-M1.md - docs/tickets/TKT-M1-03.md

Your job is to implement ONLY this ticket. Do not redesign the application or add an LLM feature.

Rules: - Never invent GW2 API fields, permissions, quotas, or behavior. Mark uncertain facts VERIFY. - Preserve the read-only boundary. Do not add gameplay or Trading Post automation. - Do not place API keys in source code, browser storage, logs, fixtures, prompts, or tests. - Keep money calculations in integer copper. - Keep external API DTOs separate from domain models. - Route GW2 requests through the single gateway defined by the architecture. - Add or update tests for every behavior change. Never weaken or delete a test just to make it pass. - Minimize unrelated file changes.

Execution protocol: 1. Inspect the current repository and relevant existing code. 2. Restate the ticket acceptance criteria in implementation terms. 3. Identify any real contradiction or missing dependency before coding. If one exists, stop and explain it rather than inventing a solution. 4. Implement the smallest coherent change satisfying the ticket. 5. Add/update unit, integration, or browser tests as appropriate. 6. Run the narrow test set first, then the relevant broader test set. 7. Check formatting/analyzers/build. 8. Review the diff for accidental scope expansion and secret leakage. 9. Finish with: files changed, tests run, results, known limitations, VERIFY items, and suggested next ticket.

Ticket-specific objective: Create reusable test helpers and a fixture policy.

Ticket-specific acceptance criteria: - Create unit, integration, and browser-test projects or equivalent harnesses. - Create fixture folders and naming conventions. - Create deterministic clock/test data helpers where needed. - Document test commands.

Ticket-specific non-goals: - Real API calls in normal test execution.

Delivery protocol (mandatory for every ticket): - Create/use a dedicated branch named `ticket/<TKT-M1-03>-<short-kebab-title>`. - Every commit for this ticket MUST start with `[TKT-M1-03]`. - Before declaring the ticket complete, push the branch and create a GitHub pull request titled `[TKT-M1-03] Short title`. - The PR body MUST identify the ticket and milestone and list the exact specification/architecture/ADR/testing/security/UX references implemented or validated, plus summary, acceptance-criteria status, validation, decisions, VERIFY items, risks/limitations, and follow-up. - Verify that the PR actually exists and report its URL. Never invent a PR URL. - Do not merge the PR. Human review and merge are required. - If GitHub CLI/authentication/permissions/remote access prevent PR creation, stop at the delivery gate and report the blocker; do not claim the ticket is complete.

You are the implementation agent for the Tyrian Ledger project.

Read first: - docs/context/permanent-context.md - docs/context/milestone-context-M1.md - docs/tickets/TKT-M1-04.md

Your job is to implement ONLY this ticket. Do not redesign the application or add an LLM feature.

Rules: - Never invent GW2 API fields, permissions, quotas, or behavior. Mark uncertain facts VERIFY. - Preserve the read-only boundary. Do not add gameplay or Trading Post automation. - Do not place API keys in source code, browser storage, logs, fixtures, prompts, or tests. - Keep money calculations in integer copper. - Keep external API DTOs separate from domain models. - Route GW2 requests through the single gateway defined by the architecture. - Add or update tests for every behavior change. Never weaken or delete a test just to make it pass. - Minimize unrelated file changes.

Execution protocol: 1. Inspect the current repository and relevant existing code. 2. Restate the ticket acceptance criteria in implementation terms. 3. Identify any real contradiction or missing dependency before coding. If one exists, stop and explain it rather than inventing a solution. 4. Implement the smallest coherent change satisfying the ticket. 5. Add/update unit, integration, or browser tests as appropriate. 6. Run the narrow test set first, then the relevant broader test set. 7. Check formatting/analyzers/build. 8. Review the diff for accidental scope expansion and secret leakage. 9. Finish with: files changed, tests run, results, known limitations, VERIFY items, and suggested next ticket.

Ticket-specific objective: Make loopback-only operation and safe HTTP defaults explicit.

Ticket-specific acceptance criteria: - Default development server binds to loopback. - Add safe response headers compatible with the local app. - Validate inbound query/body values at API boundaries. - Document how to

intentionally change the binding and why doing so is not recommended.

Ticket-specific non-goals: - LAN/public hosting support.

Delivery protocol (mandatory for every ticket): - Create/use a dedicated branch named `ticket/<TKT-M1-04>-<short-kebab-title>`. - Every commit for this ticket MUST start with `[TKT-M1-04]`. - Before declaring the ticket complete, push the branch and create a GitHub pull request titled `[TKT-M1-04] Short title`. - The PR body MUST identify the ticket and milestone and list the exact specification/architecture/ADR/testing/security/UX references implemented or validated, plus summary, acceptance-criteria status, validation, decisions, VERIFY items, risks/limitations, and follow-up. - Verify that the PR actually exists and report its URL. Never invent a PR URL. - Do not merge the PR. Human review and merge are required. - If GitHub CLI/authentication/permissions/remote access prevent PR creation, stop at the delivery gate and report the blocker; do not claim the ticket is complete.

You are the implementation agent for the Tyrian Ledger project.

Read first: - docs/context/permanent-context.md - docs/context/milestone-context-M2.md - docs/tickets/TKT-M2-01.md

Your job is to implement ONLY this ticket. Do not redesign the application or add an LLM feature.

Rules: - Never invent GW2 API fields, permissions, quotas, or behavior. Mark uncertain facts VERIFY. - Preserve the read-only boundary. Do not add gameplay or Trading Post automation. - Do not place API keys in source code, browser storage, logs, fixtures, prompts, or tests. - Keep money calculations in integer copper. - Keep external API DTOs separate from domain models. - Route GW2 requests through the single gateway defined by the architecture. - Add or update tests for every behavior change. Never weaken or delete a test just to make it pass. - Minimize unrelated file changes.

Execution protocol: 1. Inspect the current repository and relevant existing code. 2. Restate the ticket acceptance criteria in implementation terms. 3. Identify any real contradiction or missing dependency before coding. If one exists, stop and explain it rather than inventing a solution. 4. Implement the smallest coherent change satisfying the ticket. 5. Add/update unit, integration, or browser tests as appropriate. 6. Run the narrow test set first, then the relevant broader test set. 7. Check formatting/analyzers/build. 8. Review the diff for accidental scope expansion and secret leakage. 9. Finish with: files changed, tests run, results, known limitations, VERIFY items, and suggested next ticket.

Ticket-specific objective: Implement typed read-only requests for prices and listings based on verified schemas.

Ticket-specific acceptance criteria: - Only explicit GET methods are exposed. - External DTOs remain separate from domain models. - Batch IDs are supported where the verified endpoint allows them. - Unexpected fields do not crash parsing unnecessarily. - Errors map to stable error categories.

Ticket-specific non-goals: - Account/authenticated endpoints other than those explicitly needed by later tickets.

Delivery protocol (mandatory for every ticket): - Create/use a dedicated branch named `ticket/<TKT-M2-01>-<short-kebab-title>`. - Every commit for this ticket MUST start with `[TKT-M2-01]`. - Before declaring the ticket complete, push the branch and create a GitHub pull request titled `[TKT-M2-01] Short title`. - The PR body MUST identify the ticket and milestone and list the exact specification/architecture/ADR/testing/security/UX references implemented or validated, plus summary, acceptance-criteria status, validation, decisions, VERIFY items, risks/limitations, and follow-up. - Verify that the PR actually exists and report its URL. Never invent a PR URL. - Do not merge the PR. Human review and merge are required. - If GitHub CLI/authentication/permissions/remote access prevent PR creation, stop at the delivery gate and report the blocker; do not claim the ticket is complete.

You are the implementation agent for the Tyrian Ledger project.

Read first: - docs/context/permanent-context.md - docs/context/milestone-context-M2.md - docs/tickets/TKT-M2-02.md

Your job is to implement ONLY this ticket. Do not redesign the application or add an LLM feature.

Rules: - Never invent GW2 API fields, permissions, quotas, or behavior. Mark uncertain facts VERIFY. - Preserve the read-only boundary. Do not add gameplay or Trading Post automation. - Do not place API keys in source code, browser storage, logs, fixtures, prompts, or tests. - Keep money calculations in integer copper. - Keep external API DTOs separate from domain models. - Route GW2 requests through the single gateway defined by the architecture.

- Add or update tests for every behavior change. Never weaken or delete a test just to make it pass. - Minimize unrelated file changes.

Execution protocol: 1. Inspect the current repository and relevant existing code. 2. Restate the ticket acceptance criteria in implementation terms. 3. Identify any real contradiction or missing dependency before coding. If one exists, stop and explain it rather than inventing a solution. 4. Implement the smallest coherent change satisfying the ticket. 5. Add/update unit, integration, or browser tests as appropriate. 6. Run the narrow test set first, then the relevant broader test set. 7. Check formatting/analyzers/build. 8. Review the diff for accidental scope expansion and secret leakage. 9. Finish with: files changed, tests run, results, known limitations, VERIFY items, and suggested next ticket.

Ticket-specific objective: Centralize GW2 request rate management.

Ticket-specific acceptance criteria: - Requests are scheduled through one component. - Concurrent identical requests are deduplicated. - 429 responses trigger bounded backoff. - Retry rules distinguish transient from permanent failures. - Configured quotas are not hard-coded from an unverified assumption.

Ticket-specific non-goals: - Stress-testing live GW2.

Delivery protocol (mandatory for every ticket): - Create/use a dedicated branch named `ticket/<TKT-M2-02>-<short-kebab-title>`. - Every commit for this ticket MUST start with `[TKT-M2-02]`. - Before declaring the ticket complete, push the branch and create a GitHub pull request titled `[TKT-M2-02] Short title`. - The PR body MUST identify the ticket and milestone and list the exact specification/architecture/ADR/testing/security/UX references implemented or validated, plus summary, acceptance-criteria status, validation, decisions, VERIFY items, risks/limitations, and follow-up. - Verify that the PR actually exists and report its URL. Never invent a PR URL. - Do not merge the PR. Human review and merge are required. - If GitHub CLI/authentication/permissions/remote access prevent PR creation, stop at the delivery gate and report the blocker; do not claim the ticket is complete.

You are the implementation agent for the Tyrian Ledger project.

Read first: - docs/context/permanent-context.md - docs/context/milestone-context-M2.md - docs/tickets/TKT-M2-03.md

Your job is to implement ONLY this ticket. Do not redesign the application or add an LLM feature.

Rules: - Never invent GW2 API fields, permissions, quotas, or behavior. Mark uncertain facts VERIFY. - Preserve the read-only boundary. Do not add gameplay or Trading Post automation. - Do not place API keys in source code, browser storage, logs, fixtures, prompts, or tests. - Keep money calculations in integer copper. - Keep external API DTOs separate from domain models. - Route GW2 requests through the single gateway defined by the architecture. - Add or update tests for every behavior change. Never weaken or delete a test just to make it pass. - Minimize unrelated file changes.

Execution protocol: 1. Inspect the current repository and relevant existing code. 2. Restate the ticket acceptance criteria in implementation terms. 3. Identify any real contradiction or missing dependency before coding. If one exists, stop and explain it rather than inventing a solution. 4. Implement the smallest coherent change satisfying the ticket. 5. Add/update unit, integration, or browser tests as appropriate. 6. Run the narrow test set first, then the relevant broader test set. 7. Check formatting/analyzers/build. 8. Review the diff for accidental scope expansion and secret leakage. 9. Finish with: files changed, tests run, results, known limitations, VERIFY items, and suggested next ticket.

Ticket-specific objective: Cache public market responses and expose data age.

Ticket-specific acceptance criteria: - Cache entries have capture time and expiry policy. - Cache hit avoids network request. - Cache invalidation/refresh is deterministic. - Data freshness is available to analytics and UI.

Ticket-specific non-goals: - Historical market database.

Delivery protocol (mandatory for every ticket): - Create/use a dedicated branch named `ticket/<TKT-M2-03>-<short-kebab-title>`. - Every commit for this ticket MUST start with `[TKT-M2-03]`. - Before declaring the ticket complete, push the branch and create a GitHub pull request titled `[TKT-M2-03] Short title`. - The PR body MUST identify the ticket and milestone and list the exact specification/architecture/ADR/testing/security/UX references implemented or validated, plus summary, acceptance-criteria status, validation, decisions, VERIFY items, risks/limitations, and follow-up. - Verify that the PR actually exists and report its URL. Never invent a PR URL. - Do not merge the PR. Human review and merge are required. - If GitHub CLI/authentication/permissions/remote access prevent PR creation, stop at the delivery gate and report the blocker; do not claim the ticket is complete.

You are the implementation agent for the Tyrian Ledger project.

Read first: - docs/context/permanent-context.md - docs/context/milestone-context-M2.md - docs/tickets/TKT-M2-04.md

Your job is to implement ONLY this ticket. Do not redesign the application or add an LLM feature.

Rules: - Never invent GW2 API fields, permissions, quotas, or behavior. Mark uncertain facts VERIFY. - Preserve the read-only boundary. Do not add gameplay or Trading Post automation. - Do not place API keys in source code, browser storage, logs, fixtures, prompts, or tests. - Keep money calculations in integer copper. - Keep external API DTOs separate from domain models. - Route GW2 requests through the single gateway defined by the architecture. - Add or update tests for every behavior change. Never weaken or delete a test just to make it pass. - Minimize unrelated file changes.

Execution protocol: 1. Inspect the current repository and relevant existing code. 2. Restate the ticket acceptance criteria in implementation terms. 3. Identify any real contradiction or missing dependency before coding. If one exists, stop and explain it rather than inventing a solution. 4. Implement the smallest coherent change satisfying the ticket. 5. Add/update unit, integration, or browser tests as appropriate. 6. Run the narrow test set first, then the relevant broader test set. 7. Check formatting/analyzers/build. 8. Review the diff for accidental scope expansion and secret leakage. 9. Finish with: files changed, tests run, results, known limitations, VERIFY items, and suggested next ticket.

Ticket-specific objective: Make API use and cache behavior auditable.

Ticket-specific acceptance criteria: - Track request counts, cache hits/misses, latency, 429s, and parsing failures. - Never log API keys or authorization headers. - Provide a local diagnostic view or structured diagnostic endpoint that is safe to display.

Ticket-specific non-goals: - Remote telemetry.

Delivery protocol (mandatory for every ticket): - Create/use a dedicated branch named `ticket/<TKT-M2-04>-<short-kebab-title>`. - Every commit for this ticket MUST start with `[TKT-M2-04]`. - Before declaring the ticket complete, push the branch and create a GitHub pull request titled `[TKT-M2-04] Short title`. - The PR body MUST identify the ticket and milestone and list the exact specification/architecture/ADR/testing/security/UX references implemented or validated, plus summary, acceptance-criteria status, validation, decisions, VERIFY items, risks/limitations, and follow-up. - Verify that the PR actually exists and report its URL. Never invent a PR URL. - Do not merge the PR. Human review and merge are required. - If GitHub CLI/authentication/permissions/remote access prevent PR creation, stop at the delivery gate and report the blocker; do not claim the ticket is complete.

You are the implementation agent for the Tyrian Ledger project.

Read first: - docs/context/permanent-context.md - docs/context/milestone-context-M3.md - docs/tickets/TKT-M3-01.md

Your job is to implement ONLY this ticket. Do not redesign the application or add an LLM feature.

Rules: - Never invent GW2 API fields, permissions, quotas, or behavior. Mark uncertain facts VERIFY. - Preserve the read-only boundary. Do not add gameplay or Trading Post automation. - Do not place API keys in source code, browser storage, logs, fixtures, prompts, or tests. - Keep money calculations in integer copper. - Keep external API DTOs separate from domain models. - Route GW2 requests through the single gateway defined by the architecture. - Add or update tests for every behavior change. Never weaken or delete a test just to make it pass. - Minimize unrelated file changes.

Execution protocol: 1. Inspect the current repository and relevant existing code. 2. Restate the ticket acceptance criteria in implementation terms. 3. Identify any real contradiction or missing dependency before coding. If one exists, stop and explain it rather than inventing a solution. 4. Implement the smallest coherent change satisfying the ticket. 5. Add/update unit, integration, or browser tests as appropriate. 6. Run the narrow test set first, then the relevant broader test set. 7. Check formatting/analyzers/build. 8. Review the diff for accidental scope expansion and secret leakage. 9. Finish with: files changed, tests run, results, known limitations, VERIFY items, and suggested next ticket.

Ticket-specific objective: Create exact copper arithmetic and centralized transaction-fee policy.

Ticket-specific acceptance criteria: - Money is represented in integer copper. - No floating-point arithmetic is used for money calculations. - Fee policy is isolated and configurable. - Profit formulas document scenario semantics.

Ticket-specific non-goals: - Hard-coding unexplained fee constants throughout the application.

Delivery protocol (mandatory for every ticket): - Create/use a dedicated branch named `ticket/<TKT-M3-01>-<short-kebab-title>`. - Every commit for this ticket MUST start with `[TKT-M3-01] Short title`. - Before declaring the ticket complete, push the branch and create a GitHub pull request titled `[TKT-M3-01] Short title`. - The PR body MUST identify the ticket and milestone and list the exact specification/architecture/ADR/testing/security/UX references implemented or validated, plus summary, acceptance-criteria status, validation, decisions, VERIFY items, risks/limitations, and follow-up. - Verify that the PR actually exists and report its URL. Never invent a PR URL. - Do not merge the PR. Human review and merge are required. - If GitHub CLI/authentication/permissions/remote access prevent PR creation, stop at the delivery gate and report the blocker; do not claim the ticket is complete.

You are the implementation agent for the Tyrian Ledger project.

Read first: - docs/context/permanent-context.md - docs/context/milestone-context-M3.md - docs/tickets/TKT-M3-02.md

Your job is to implement ONLY this ticket. Do not redesign the application or add an LLM feature.

Rules: - Never invent GW2 API fields, permissions, quotas, or behavior. Mark uncertain facts VERIFY. - Preserve the read-only boundary. Do not add gameplay or Trading Post automation. - Do not place API keys in source code, browser storage, logs, fixtures, prompts, or tests. - Keep money calculations in integer copper. - Keep external API DTOs separate from domain models. - Route GW2 requests through the single gateway defined by the architecture. - Add or update tests for every behavior change. Never weaken or delete a test just to make it pass. - Minimize unrelated file changes.

Execution protocol: 1. Inspect the current repository and relevant existing code. 2. Restate the ticket acceptance criteria in implementation terms. 3. Identify any real contradiction or missing dependency before coding. If one exists, stop and explain it rather than inventing a solution. 4. Implement the smallest coherent change satisfying the ticket. 5. Add/update unit, integration, or browser tests as appropriate. 6. Run the narrow test set first, then the relevant broader test set. 7. Check formatting/analyzers/build. 8. Review the diff for accidental scope expansion and secret leakage. 9. Finish with: files changed, tests run, results, known limitations, VERIFY items, and suggested next ticket.

Ticket-specific objective: Calculate acquisition and liquidation scenarios across order-book levels.

Ticket-specific acceptance criteria: - Simulate quantity across multiple buy/sell levels. - Calculate weighted average execution price and price impact. - Handle insufficient depth explicitly. - Return transparent assumptions and remaining quantity.

Ticket-specific non-goals: - Claiming guaranteed real-world fill behavior.

Delivery protocol (mandatory for every ticket): - Create/use a dedicated branch named `ticket/<TKT-M3-02>-<short-kebab-title>`. - Every commit for this ticket MUST start with `[TKT-M3-02]`. - Before declaring the ticket complete, push the branch and create a GitHub pull request titled `[TKT-M3-02] Short title`. - The PR body MUST identify the ticket and milestone and list the exact specification/architecture/ADR/testing/security/UX references implemented or validated, plus summary, acceptance-criteria status, validation, decisions, VERIFY items, risks/limitations, and follow-up. - Verify that the PR actually exists and report its URL. Never invent a PR URL. - Do not merge the PR. Human review and merge are required. - If GitHub CLI/authentication/permissions/remote access prevent PR creation, stop at the delivery gate and report the blocker; do not claim the ticket is complete.

You are the implementation agent for the Tyrian Ledger project.

Read first: - docs/context/permanent-context.md - docs/context/milestone-context-M3.md - docs/tickets/TKT-M3-03.md

Your job is to implement ONLY this ticket. Do not redesign the application or add an LLM feature.

Rules: - Never invent GW2 API fields, permissions, quotas, or behavior. Mark uncertain facts VERIFY. - Preserve the read-only boundary. Do not add gameplay or Trading Post automation. - Do not place API keys in source code, browser storage, logs, fixtures, prompts, or tests. - Keep money calculations in integer copper. - Keep external API

DTOs separate from domain models. - Route GW2 requests through the single gateway defined by the architecture. - Add or update tests for every behavior change. Never weaken or delete a test just to make it pass. - Minimize unrelated file changes.

Execution protocol: 1. Inspect the current repository and relevant existing code. 2. Restate the ticket acceptance criteria in implementation terms. 3. Identify any real contradiction or missing dependency before coding. If one exists, stop and explain it rather than inventing a solution. 4. Implement the smallest coherent change satisfying the ticket. 5. Add/update unit, integration, or browser tests as appropriate. 6. Run the narrow test set first, then the relevant broader test set. 7. Check formatting/analyzers/build. 8. Review the diff for accidental scope expansion and secret leakage. 9. Finish with: files changed, tests run, results, known limitations, VERIFY items, and suggested next ticket.

Ticket-specific objective: Generate deterministic flip scenarios and a liquidity proxy.

Ticket-specific acceptance criteria: - Compute modeled net profit, ROI, capital required, price impact, and liquidity metrics. - Support configurable minimum profit and capital constraints. - Mark missing/stale data as unusable or lower confidence according to policy. - Each result explains the scenario used.

Ticket-specific non-goals: - Automated orders.

Delivery protocol (mandatory for every ticket): - Create/use a dedicated branch named `ticket/<TKT-M3-03>-<short-kebab-title>`. - Every commit for this ticket MUST start with `[TKT-M3-03]`. - Before declaring the ticket complete, push the branch and create a GitHub pull request titled `[TKT-M3-03] Short title`. - The PR body MUST identify the ticket and milestone and list the exact specification/architecture/ADR/testing/security/UX references implemented or validated, plus summary, acceptance-criteria status, validation, decisions, VERIFY items, risks/limitations, and follow-up. - Verify that the PR actually exists and report its URL. Never invent a PR URL. - Do not merge the PR. Human review and merge are required. - If GitHub CLI/authentication/permissions/remote access prevent PR creation, stop at the delivery gate and report the blocker; do not claim the ticket is complete.

You are the implementation agent for the Tyrian Ledger project.

Read first: - docs/context/permanent-context.md - docs/context/milestone-context-M3.md - docs/tickets/TKT-M3-04.md

Your job is to implement ONLY this ticket. Do not redesign the application or add an LLM feature.

Rules: - Never invent GW2 API fields, permissions, quotas, or behavior. Mark uncertain facts VERIFY. - Preserve the read-only boundary. Do not add gameplay or Trading Post automation. - Do not place API keys in source code, browser storage, logs, fixtures, prompts, or tests. - Keep money calculations in integer copper. - Keep external API DTOs separate from domain models. - Route GW2 requests through the single gateway defined by the architecture. - Add or update tests for every behavior change. Never weaken or delete a test just to make it pass. - Minimize unrelated file changes.

Execution protocol: 1. Inspect the current repository and relevant existing code. 2. Restate the ticket acceptance criteria in implementation terms. 3. Identify any real contradiction or missing dependency before coding. If one exists, stop and explain it rather than inventing a solution. 4. Implement the smallest coherent change satisfying the ticket. 5. Add/update unit, integration, or browser tests as appropriate. 6. Run the narrow test set first, then the relevant broader test set. 7. Check formatting/analyzers/build. 8. Review the diff for accidental scope expansion and secret leakage. 9. Finish with: files changed, tests run, results, known limitations, VERIFY items, and suggested next ticket.

Ticket-specific objective: Rank opportunities using transparent weights and penalties.

Ticket-specific acceptance criteria: - Score is deterministic for identical inputs/configuration. - Weights can be configured without changing code. - Score includes at least profit, capital efficiency, liquidity, freshness, risk, and complexity where applicable. - UI-facing explanation metadata is produced.

Ticket-specific non-goals: - LLM-generated ranking.

Delivery protocol (mandatory for every ticket): - Create/use a dedicated branch named `ticket/<TKT-M3-04>-<short-kebab-title>`. - Every commit for this ticket MUST start with `[TKT-M3-04]`. - Before declaring the ticket complete, push the branch and create a GitHub pull request titled `[TKT-M3-04] Short title`. - The PR body MUST identify the ticket and milestone and list the exact specification/architecture/ADR/testing/security/UX references implemented or validated, plus summary, acceptance-criteria status, validation, decisions, VERIFY items, risks/limitations, and follow-up. - Verify that the PR actually exists and report its URL. Never invent a PR URL. - Do not merge the

PR. Human review and merge are required. - If GitHub CLI/authentication/permissions/remote access prevent PR creation, stop at the delivery gate and report the blocker; do not claim the ticket is complete.

You are the implementation agent for the Tyrian Ledger project.

Read first: - docs/context/permanent-context.md - docs/context/milestone-context-M4.md - docs/tickets/TKT-M4-01.md

Your job is to implement ONLY this ticket. Do not redesign the application or add an LLM feature.

Rules: - Never invent GW2 API fields, permissions, quotas, or behavior. Mark uncertain facts VERIFY. - Preserve the read-only boundary. Do not add gameplay or Trading Post automation. - Do not place API keys in source code, browser storage, logs, fixtures, prompts, or tests. - Keep money calculations in integer copper. - Keep external API DTOs separate from domain models. - Route GW2 requests through the single gateway defined by the architecture. - Add or update tests for every behavior change. Never weaken or delete a test just to make it pass. - Minimize unrelated file changes.

Execution protocol: 1. Inspect the current repository and relevant existing code. 2. Restate the ticket acceptance criteria in implementation terms. 3. Identify any real contradiction or missing dependency before coding. If one exists, stop and explain it rather than inventing a solution. 4. Implement the smallest coherent change satisfying the ticket. 5. Add/update unit, integration, or browser tests as appropriate. 6. Run the narrow test set first, then the relevant broader test set. 7. Check formatting/analyzers/build. 8. Review the diff for accidental scope expansion and secret leakage. 9. Finish with: files changed, tests run, results, known limitations, VERIFY items, and suggested next ticket.

Ticket-specific objective: Build the main desktop dashboard around deterministic opportunities.

Ticket-specific acceptance criteria: - Show ranked opportunities with capital, modeled profit, ROI, liquidity, risk/confidence, and data age. - Provide filters for capital, minimum profit, strategy, risk, and freshness. - No unsupported guarantee language. - Empty/loading/error states are explicit.

Ticket-specific non-goals: - Mobile-first UI.

Delivery protocol (mandatory for every ticket): - Create/use a dedicated branch named `ticket/<TKT-M4-01>-<short-kebab-title>`. - Every commit for this ticket MUST start with `[TKT-M4-01]`. - Before declaring the ticket complete, push the branch and create a GitHub pull request titled `[TKT-M4-01] Short title`. - The PR body MUST identify the ticket and milestone and list the exact specification/architecture/ADR/testing/security/UX references implemented or validated, plus summary, acceptance-criteria status, validation, decisions, VERIFY items, risks/limitations, and follow-up. - Verify that the PR actually exists and report its URL. Never invent a PR URL. - Do not merge the PR. Human review and merge are required. - If GitHub CLI/authentication/permissions/remote access prevent PR creation, stop at the delivery gate and report the blocker; do not claim the ticket is complete.

You are the implementation agent for the Tyrian Ledger project.

Read first: - docs/context/permanent-context.md - docs/context/milestone-context-M4.md - docs/tickets/TKT-M4-02.md

Your job is to implement ONLY this ticket. Do not redesign the application or add an LLM feature.

Rules: - Never invent GW2 API fields, permissions, quotas, or behavior. Mark uncertain facts VERIFY. - Preserve the read-only boundary. Do not add gameplay or Trading Post automation. - Do not place API keys in source code, browser storage, logs, fixtures, prompts, or tests. - Keep money calculations in integer copper. - Keep external API DTOs separate from domain models. - Route GW2 requests through the single gateway defined by the architecture. - Add or update tests for every behavior change. Never weaken or delete a test just to make it pass. - Minimize unrelated file changes.

Execution protocol: 1. Inspect the current repository and relevant existing code. 2. Restate the ticket acceptance criteria in implementation terms. 3. Identify any real contradiction or missing dependency before coding. If one exists, stop and explain it rather than inventing a solution. 4. Implement the smallest coherent change satisfying the ticket. 5. Add/update unit, integration, or browser tests as appropriate. 6. Run the narrow test set first, then the relevant broader test set. 7. Check formatting/analyzers/build. 8. Review the diff for accidental scope expansion and secret leakage. 9. Finish with: files changed, tests run, results, known limitations, VERIFY items, and suggested next ticket.

Ticket-specific objective: Expose calculation details and assumptions.

Ticket-specific acceptance criteria: - Show acquisition/exit assumptions, fees, modeled profit, ROI, capital, order-book impact, liquidity, and data age. - Provide a human-readable calculation breakdown. - Clearly distinguish scenario values from actual outcomes.

Ticket-specific non-goals: - Executing actions from the detail page.

Delivery protocol (mandatory for every ticket): - Create/use a dedicated branch named `ticket/<TKT-M4-02>-<short-kebab-title>`. - Every commit for this ticket MUST start with `[TKT-M4-02]`. - Before declaring the ticket complete, push the branch and create a GitHub pull request titled `[TKT-M4-02] Short title`. - The PR body MUST identify the ticket and milestone and list the exact specification/architecture/ADR/testing/security/UX references implemented or validated, plus summary, acceptance-criteria status, validation, decisions, VERIFY items, risks/limitations, and follow-up. - Verify that the PR actually exists and report its URL. Never invent a PR URL. - Do not merge the PR. Human review and merge are required. - If GitHub CLI/authentication/permissions/remote access prevent PR creation, stop at the delivery gate and report the blocker; do not claim the ticket is complete.

You are the implementation agent for the Tyrian Ledger project.

Read first: - docs/context/permanent-context.md - docs/context/milestone-context-M4.md - docs/tickets/TKT-M4-03.md

Your job is to implement ONLY this ticket. Do not redesign the application or add an LLM feature.

Rules: - Never invent GW2 API fields, permissions, quotas, or behavior. Mark uncertain facts VERIFY. - Preserve the read-only boundary. Do not add gameplay or Trading Post automation. - Do not place API keys in source code, browser storage, logs, fixtures, prompts, or tests. - Keep money calculations in integer copper. - Keep external API DTOs separate from domain models. - Route GW2 requests through the single gateway defined by the architecture. - Add or update tests for every behavior change. Never weaken or delete a test just to make it pass. - Minimize unrelated file changes.

Execution protocol: 1. Inspect the current repository and relevant existing code. 2. Restate the ticket acceptance criteria in implementation terms. 3. Identify any real contradiction or missing dependency before coding. If one exists, stop and explain it rather than inventing a solution. 4. Implement the smallest coherent change satisfying the ticket. 5. Add/update unit, integration, or browser tests as appropriate. 6. Run the narrow test set first, then the relevant broader test set. 7. Check formatting/analyzers/build. 8. Review the diff for accidental scope expansion and secret leakage. 9. Finish with: files changed, tests run, results, known limitations, VERIFY items, and suggested next ticket.

Ticket-specific objective: Persist user capital and opportunity preferences locally.

Ticket-specific acceptance criteria: - Store capital, minimum profit, risk preference, strategy preference, and allocation constraints locally. - Validate numeric ranges and sensible defaults. - Changing preferences re-ranks or filters results deterministically.

Ticket-specific non-goals: - Remote user accounts.

Delivery protocol (mandatory for every ticket): - Create/use a dedicated branch named `ticket/<TKT-M4-03>-<short-kebab-title>`. - Every commit for this ticket MUST start with `[TKT-M4-03]`. - Before declaring the ticket complete, push the branch and create a GitHub pull request titled `[TKT-M4-03] Short title`. - The PR body MUST identify the ticket and milestone and list the exact specification/architecture/ADR/testing/security/UX references implemented or validated, plus summary, acceptance-criteria status, validation, decisions, VERIFY items, risks/limitations, and follow-up. - Verify that the PR actually exists and report its URL. Never invent a PR URL. - Do not merge the PR. Human review and merge are required. - If GitHub CLI/authentication/permissions/remote access prevent PR creation, stop at the delivery gate and report the blocker; do not claim the ticket is complete.

You are the implementation agent for the Tyrian Ledger project.

Read first: - docs/context/permanent-context.md - docs/context/milestone-context-M4.md - docs/tickets/TKT-M4-04.md

Your job is to implement ONLY this ticket. Do not redesign the application or add an LLM feature.

Rules: - Never invent GW2 API fields, permissions, quotas, or behavior. Mark uncertain facts VERIFY. - Preserve the read-only boundary. Do not add gameplay or Trading Post automation. - Do not place API keys in source code, browser storage, logs, fixtures, prompts, or tests. - Keep money calculations in integer copper. - Keep external API DTOs separate from domain models. - Route GW2 requests through the single gateway defined by the architecture. - Add or update tests for every behavior change. Never weaken or delete a test just to make it pass. - Minimize unrelated file changes.

Execution protocol: 1. Inspect the current repository and relevant existing code. 2. Restate the ticket acceptance criteria in implementation terms. 3. Identify any real contradiction or missing dependency before coding. If one exists, stop and explain it rather than inventing a solution. 4. Implement the smallest coherent change satisfying the ticket. 5. Add/update unit, integration, or browser tests as appropriate. 6. Run the narrow test set first, then the relevant broader test set. 7. Check formatting/analyzers/build. 8. Review the diff for accidental scope expansion and secret leakage. 9. Finish with: files changed, tests run, results, known limitations, VERIFY items, and suggested next ticket.

Ticket-specific objective: Turn constraints into a practical shortlist without pretending to know execution time.

Ticket-specific acceptance criteria: - Support effort categories: very low, low, medium, high, ongoing/patient. - Planner respects capital/risk/preferences. - Planner produces an ordered session shortlist. - UI states that categories are approximations, not time guarantees.

Ticket-specific non-goals: - Precise execution-time prediction.

Delivery protocol (mandatory for every ticket): - Create/use a dedicated branch named `ticket/<TKT-M4-04>-<short-kebab-title>`. - Every commit for this ticket MUST start with `[TKT-M4-04]`. - Before declaring the ticket complete, push the branch and create a GitHub pull request titled `[TKT-M4-04] Short title`. - The PR body MUST identify the ticket and milestone and list the exact specification/architecture/ADR/testing/security/UX references implemented or validated, plus summary, acceptance-criteria status, validation, decisions, VERIFY items, risks/limitations, and follow-up. - Verify that the PR actually exists and report its URL. Never invent a PR URL. - Do not merge the PR. Human review and merge are required. - If GitHub CLI/authentication/permissions/remote access prevent PR creation, stop at the delivery gate and report the blocker; do not claim the ticket is complete.

You are the implementation agent for the Tyrian Ledger project.

Read first: - docs/context/permanent-context.md - docs/context/milestone-context-M5.md - docs/tickets/TKT-M5-01.md

Your job is to implement ONLY this ticket. Do not redesign the application or add an LLM feature.

Rules: - Never invent GW2 API fields, permissions, quotas, or behavior. Mark uncertain facts VERIFY. - Preserve the read-only boundary. Do not add gameplay or Trading Post automation. - Do not place API keys in source code, browser storage, logs, fixtures, prompts, or tests. - Keep money calculations in integer copper. - Keep external API DTOs separate from domain models. - Route GW2 requests through the single gateway defined by the architecture. - Add or update tests for every behavior change. Never weaken or delete a test just to make it pass. - Minimize unrelated file changes.

Execution protocol: 1. Inspect the current repository and relevant existing code. 2. Restate the ticket acceptance criteria in implementation terms. 3. Identify any real contradiction or missing dependency before coding. If one exists, stop and explain it rather than inventing a solution. 4. Implement the smallest coherent change satisfying the ticket. 5. Add/update unit, integration, or browser tests as appropriate. 6. Run the narrow test set first, then the relevant broader test set. 7. Check formatting/analyzers/build. 8. Review the diff for accidental scope expansion and secret leakage. 9. Finish with: files changed, tests run, results, known limitations, VERIFY items, and suggested next ticket.

Ticket-specific objective: Validate an API key and expose only permitted features.

Ticket-specific acceptance criteria: - Use tokeninfo or the verified equivalent to validate the key. - Display safe permission status. - Never return the key to the browser. - Disable unsupported account features when scopes are missing. - Render token metadata such as key name as text, not HTML.

Ticket-specific non-goals: - Storing the API key in frontend localStorage.

Delivery protocol (mandatory for every ticket): - Create/use a dedicated branch named `ticket/<TKT-M5-01>-<short-kebab-title>`. - Every commit for this ticket MUST start with `[TKT-M5-01]`. - Before declaring the ticket complete, push the branch and create a GitHub pull request titled `[TKT-M5-01] Short title`. - The PR body MUST identify the

ticket and milestone and list the exact specification/architecture/ADR/testing/security/UX references implemented or validated, plus summary, acceptance-criteria status, validation, decisions, VERIFY items, risks/limitations, and follow-up. - Verify that the PR actually exists and report its URL. Never invent a PR URL. - Do not merge the PR. Human review and merge are required. - If GitHub CLI/authentication/permissions/remote access prevent PR creation, stop at the delivery gate and report the blocker; do not claim the ticket is complete.

You are the implementation agent for the Tyrian Ledger project.

Read first: - docs/context/permanent-context.md - docs/context/milestone-context-M5.md - docs/tickets/TKT-M5-02.md

Your job is to implement ONLY this ticket. Do not redesign the application or add an LLM feature.

Rules: - Never invent GW2 API fields, permissions, quotas, or behavior. Mark uncertain facts VERIFY. - Preserve the read-only boundary. Do not add gameplay or Trading Post automation. - Do not place API keys in source code, browser storage, logs, fixtures, prompts, or tests. - Keep money calculations in integer copper. - Keep external API DTOs separate from domain models. - Route GW2 requests through the single gateway defined by the architecture. - Add or update tests for every behavior change. Never weaken or delete a test just to make it pass. - Minimize unrelated file changes.

Execution protocol: 1. Inspect the current repository and relevant existing code. 2. Restate the ticket acceptance criteria in implementation terms. 3. Identify any real contradiction or missing dependency before coding. If one exists, stop and explain it rather than inventing a solution. 4. Implement the smallest coherent change satisfying the ticket. 5. Add/update unit, integration, or browser tests as appropriate. 6. Run the narrow test set first, then the relevant broader test set. 7. Check formatting/analyzers/build. 8. Review the diff for accidental scope expansion and secret leakage. 9. Finish with: files changed, tests run, results, known limitations, VERIFY items, and suggested next ticket.

Ticket-specific objective: Add bank/materials and relevant character data with caching and local scoping.

Ticket-specific acceptance criteria: - Fetch only endpoints required for enabled account features. - Cache account snapshots separately from public market data. - Associate snapshots with a local account profile identifier. - Handle missing permissions and partial data gracefully.

Ticket-specific non-goals: - Persisting unnecessary full raw payloads indefinitely.

Delivery protocol (mandatory for every ticket): - Create/use a dedicated branch named `ticket/<TKT-M5-02>-<short-kebab-title>`. - Every commit for this ticket MUST start with `[TKT-M5-02]`. - Before declaring the ticket complete, push the branch and create a GitHub pull request titled `[TKT-M5-02] Short title`. - The PR body MUST identify the ticket and milestone and list the exact specification/architecture/ADR/testing/security/UX references implemented or validated, plus summary, acceptance-criteria status, validation, decisions, VERIFY items, risks/limitations, and follow-up. - Verify that the PR actually exists and report its URL. Never invent a PR URL. - Do not merge the PR. Human review and merge are required. - If GitHub CLI/authentication/permissions/remote access prevent PR creation, stop at the delivery gate and report the blocker; do not claim the ticket is complete.

You are the implementation agent for the Tyrian Ledger project.

Read first: - docs/context/permanent-context.md - docs/context/milestone-context-M5.md - docs/tickets/TKT-M5-03.md

Your job is to implement ONLY this ticket. Do not redesign the application or add an LLM feature.

Rules: - Never invent GW2 API fields, permissions, quotas, or behavior. Mark uncertain facts VERIFY. - Preserve the read-only boundary. Do not add gameplay or Trading Post automation. - Do not place API keys in source code, browser storage, logs, fixtures, prompts, or tests. - Keep money calculations in integer copper. - Keep external API DTOs separate from domain models. - Route GW2 requests through the single gateway defined by the architecture. - Add or update tests for every behavior change. Never weaken or delete a test just to make it pass. - Minimize unrelated file changes.

Execution protocol: 1. Inspect the current repository and relevant existing code. 2. Restate the ticket acceptance criteria in implementation terms. 3. Identify any real contradiction or missing dependency before coding. If one exists, stop and explain it rather than inventing a solution. 4. Implement the smallest coherent change satisfying the ticket.

5. Add/update unit, integration, or browser tests as appropriate. 6. Run the narrow test set first, then the relevant broader test set. 7. Check formatting/analyzers/build. 8. Review the diff for accidental scope expansion and secret leakage. 9. Finish with: files changed, tests run, results, known limitations, VERIFY items, and suggested next ticket.

Ticket-specific objective: Value owned materials economically and compare with buying them.

Ticket-specific acceptance criteria: - Owned materials are never assumed free. - Compute a realizable economic value using configured market evidence. - Support owned/buy/mixed strategies where data is sufficient. - Flag bound/unavailable/non-sellable items where relevant data exists.

Ticket-specific non-goals: - Assuming all inventory is sellable merely because it has an item ID.

Delivery protocol (mandatory for every ticket): - Create/use a dedicated branch named `ticket/<TKT-M5-03>-<short-kebab-title>`. - Every commit for this ticket MUST start with `[TKT-M5-03]`. - Before declaring the ticket complete, push the branch and create a GitHub pull request titled `[TKT-M5-03] Short title`. - The PR body MUST identify the ticket and milestone and list the exact specification/architecture/ADR/testing/security/UX references implemented or validated, plus summary, acceptance-criteria status, validation, decisions, VERIFY items, risks/limitations, and follow-up. - Verify that the PR actually exists and report its URL. Never invent a PR URL. - Do not merge the PR. Human review and merge are required. - If GitHub CLI/authentication/permissions/remote access prevent PR creation, stop at the delivery gate and report the blocker; do not claim the ticket is complete.

You are the implementation agent for the Tyrian Ledger project.

Read first: - docs/context/permanent-context.md - docs/context/milestone-context-M5.md - docs/tickets/TKT-M5-04.md

Your job is to implement ONLY this ticket. Do not redesign the application or add an LLM feature.

Rules: - Never invent GW2 API fields, permissions, quotas, or behavior. Mark uncertain facts VERIFY. - Preserve the read-only boundary. Do not add gameplay or Trading Post automation. - Do not place API keys in source code, browser storage, logs, fixtures, prompts, or tests. - Keep money calculations in integer copper. - Keep external API DTOs separate from domain models. - Route GW2 requests through the single gateway defined by the architecture. - Add or update tests for every behavior change. Never weaken or delete a test just to make it pass. - Minimize unrelated file changes.

Execution protocol: 1. Inspect the current repository and relevant existing code. 2. Restate the ticket acceptance criteria in implementation terms. 3. Identify any real contradiction or missing dependency before coding. If one exists, stop and explain it rather than inventing a solution. 4. Implement the smallest coherent change satisfying the ticket. 5. Add/update unit, integration, or browser tests as appropriate. 6. Run the narrow test set first, then the relevant broader test set. 7. Check formatting/analyzers/build. 8. Review the diff for accidental scope expansion and secret leakage. 9. Finish with: files changed, tests run, results, known limitations, VERIFY items, and suggested next ticket.

Ticket-specific objective: Analyze profitable crafting paths with feasibility constraints and controlled recursion.

Ticket-specific acceptance criteria: - Recipe graph represents output/ingredient relationships. - Apply discipline/rating and recipe availability constraints when verified data exists. - Search detects cycles and has configurable depth/candidate limits. - Memoization prevents repeated subproblem explosion. - Analysis reports truncation/unknowns explicitly. - Compare purchase cost versus owned-material opportunity cost.

Ticket-specific non-goals: - Exhaustive unlimited world optimization.

Delivery protocol (mandatory for every ticket): - Create/use a dedicated branch named `ticket/<TKT-M5-04>-<short-kebab-title>`. - Every commit for this ticket MUST start with `[TKT-M5-04]`. - Before declaring the ticket complete, push the branch and create a GitHub pull request titled `[TKT-M5-04] Short title`. - The PR body MUST identify the ticket and milestone and list the exact specification/architecture/ADR/testing/security/UX references implemented or validated, plus summary, acceptance-criteria status, validation, decisions, VERIFY items, risks/limitations, and follow-up. - Verify that the PR actually exists and report its URL. Never invent a PR URL. - Do not merge the PR. Human review and merge are required. - If GitHub CLI/authentication/permissions/remote access prevent PR creation, stop at the delivery gate and report the blocker; do not claim the ticket is complete.

You are the implementation agent for the Tyrian Ledger project.

Read first: - docs/context/permanent-context.md - docs/context/milestone-context-M6.md - docs/tickets/TKT-M6-01.md

Your job is to implement ONLY this ticket. Do not redesign the application or add an LLM feature.

Rules: - Never invent GW2 API fields, permissions, quotas, or behavior. Mark uncertain facts VERIFY. - Preserve the read-only boundary. Do not add gameplay or Trading Post automation. - Do not place API keys in source code, browser storage, logs, fixtures, prompts, or tests. - Keep money calculations in integer copper. - Keep external API DTOs separate from domain models. - Route GW2 requests through the single gateway defined by the architecture. - Add or update tests for every behavior change. Never weaken or delete a test just to make it pass. - Minimize unrelated file changes.

Execution protocol: 1. Inspect the current repository and relevant existing code. 2. Restate the ticket acceptance criteria in implementation terms. 3. Identify any real contradiction or missing dependency before coding. If one exists, stop and explain it rather than inventing a solution. 4. Implement the smallest coherent change satisfying the ticket. 5. Add/update unit, integration, or browser tests as appropriate. 6. Run the narrow test set first, then the relevant broader test set. 7. Check formatting/analyzers/build. 8. Review the diff for accidental scope expansion and secret leakage. 9. Finish with: files changed, tests run, results, known limitations, VERIFY items, and suggested next ticket.

Ticket-specific objective: Store saved/planned operations and their calculation context.

Ticket-specific acceptance criteria: - Store operations with timestamps and calculation/configuration version identifiers. - Preserve the scenario assumptions used at save time. - Allow user to mark status such as planned/in-progress/completed/cancelled.

Ticket-specific non-goals: - Syncing history to a cloud service.

Delivery protocol (mandatory for every ticket): - Create/use a dedicated branch named `ticket/<TKT-M6-01>-<short-kebab-title>`. - Every commit for this ticket MUST start with `[TKT-M6-01]`. - Before declaring the ticket complete, push the branch and create a GitHub pull request titled `[TKT-M6-01] Short title`. - The PR body MUST identify the ticket and milestone and list the exact specification/architecture/ADR/testing/security/UX references implemented or validated, plus summary, acceptance-criteria status, validation, decisions, VERIFY items, risks/limitations, and follow-up. - Verify that the PR actually exists and report its URL. Never invent a PR URL. - Do not merge the PR. Human review and merge are required. - If GitHub CLI/authentication/permissions/remote access prevent PR creation, stop at the delivery gate and report the blocker; do not claim the ticket is complete.

You are the implementation agent for the Tyrian Ledger project.

Read first: - docs/context/permanent-context.md - docs/context/milestone-context-M6.md - docs/tickets/TKT-M6-02.md

Your job is to implement ONLY this ticket. Do not redesign the application or add an LLM feature.

Rules: - Never invent GW2 API fields, permissions, quotas, or behavior. Mark uncertain facts VERIFY. - Preserve the read-only boundary. Do not add gameplay or Trading Post automation. - Do not place API keys in source code, browser storage, logs, fixtures, prompts, or tests. - Keep money calculations in integer copper. - Keep external API DTOs separate from domain models. - Route GW2 requests through the single gateway defined by the architecture. - Add or update tests for every behavior change. Never weaken or delete a test just to make it pass. - Minimize unrelated file changes.

Execution protocol: 1. Inspect the current repository and relevant existing code. 2. Restate the ticket acceptance criteria in implementation terms. 3. Identify any real contradiction or missing dependency before coding. If one exists, stop and explain it rather than inventing a solution. 4. Implement the smallest coherent change satisfying the ticket. 5. Add/update unit, integration, or browser tests as appropriate. 6. Run the narrow test set first, then the relevant broader test set. 7. Check formatting/analyzers/build. 8. Review the diff for accidental scope expansion and secret leakage. 9. Finish with: files changed, tests run, results, known limitations, VERIFY items, and suggested next ticket.

Ticket-specific objective: Calculate realized profit separately from unrealized P/L.

Ticket-specific acceptance criteria: - Realized profit uses recorded actual acquisition/sale values and fees. - Unrealized value is separately labeled. - Incomplete operations are not silently counted as realized profit. - Profit statistics are reproducible from stored records.

Ticket-specific non-goals: - Claiming lifetime profit before local history starts.

Delivery protocol (mandatory for every ticket): - Create/use a dedicated branch named `ticket/<TKT-M6-02>-<short-kebab-title>`. - Every commit for this ticket MUST start with `[TKT-M6-02]`. - Before declaring the ticket complete, push the branch and create a GitHub pull request titled `[TKT-M6-02] Short title`. - The PR body MUST identify the ticket and milestone and list the exact specification/architecture/ADR/testing/security/UX references implemented or validated, plus summary, acceptance-criteria status, validation, decisions, VERIFY items, risks/limitations, and follow-up. - Verify that the PR actually exists and report its URL. Never invent a PR URL. - Do not merge the PR. Human review and merge are required. - If GitHub CLI/authentication/permissions/remote access prevent PR creation, stop at the delivery gate and report the blocker; do not claim the ticket is complete.

You are the implementation agent for the Tyrian Ledger project.

Read first: - docs/context/permanent-context.md - docs/context/milestone-context-M6.md - docs/tickets/TKT-M6-03.md

Your job is to implement ONLY this ticket. Do not redesign the application or add an LLM feature.

Rules: - Never invent GW2 API fields, permissions, quotas, or behavior. Mark uncertain facts VERIFY. - Preserve the read-only boundary. Do not add gameplay or Trading Post automation. - Do not place API keys in source code, browser storage, logs, fixtures, prompts, or tests. - Keep money calculations in integer copper. - Keep external API DTOs separate from domain models. - Route GW2 requests through the single gateway defined by the architecture. - Add or update tests for every behavior change. Never weaken or delete a test just to make it pass. - Minimize unrelated file changes.

Execution protocol: 1. Inspect the current repository and relevant existing code. 2. Restate the ticket acceptance criteria in implementation terms. 3. Identify any real contradiction or missing dependency before coding. If one exists, stop and explain it rather than inventing a solution. 4. Implement the smallest coherent change satisfying the ticket. 5. Add/update unit, integration, or browser tests as appropriate. 6. Run the narrow test set first, then the relevant broader test set. 7. Check formatting/analyzers/build. 8. Review the diff for accidental scope expansion and secret leakage. 9. Finish with: files changed, tests run, results, known limitations, VERIFY items, and suggested next ticket.

Ticket-specific objective: Show useful local statistics without overstating their meaning.

Ticket-specific acceptance criteria: - Show operation count, realized profit since first recorded use, average modeled/realized metrics where justified, and completion rate where defined. - Clearly show the historical coverage period. - Provide empty-state messaging for insufficient history.

Ticket-specific non-goals: - Backfilling unknown lifetime history.

Delivery protocol (mandatory for every ticket): - Create/use a dedicated branch named `ticket/<TKT-M6-03>-<short-kebab-title>`. - Every commit for this ticket MUST start with `[TKT-M6-03]`. - Before declaring the ticket complete, push the branch and create a GitHub pull request titled `[TKT-M6-03] Short title`. - The PR body MUST identify the ticket and milestone and list the exact specification/architecture/ADR/testing/security/UX references implemented or validated, plus summary, acceptance-criteria status, validation, decisions, VERIFY items, risks/limitations, and follow-up. - Verify that the PR actually exists and report its URL. Never invent a PR URL. - Do not merge the PR. Human review and merge are required. - If GitHub CLI/authentication/permissions/remote access prevent PR creation, stop at the delivery gate and report the blocker; do not claim the ticket is complete.

You are the implementation agent for the Tyrian Ledger project.

Read first: - docs/context/permanent-context.md - docs/context/milestone-context-M6.md - docs/tickets/TKT-M6-04.md

Your job is to implement ONLY this ticket. Do not redesign the application or add an LLM feature.

Rules: - Never invent GW2 API fields, permissions, quotas, or behavior. Mark uncertain facts VERIFY. - Preserve the read-only boundary. Do not add gameplay or Trading Post automation. - Do not place API keys in source code, browser storage, logs, fixtures, prompts, or tests. - Keep money calculations in integer copper. - Keep external API DTOs separate from domain models. - Route GW2 requests through the single gateway defined by the architecture.

- Add or update tests for every behavior change. Never weaken or delete a test just to make it pass. - Minimize unrelated file changes.

Execution protocol: 1. Inspect the current repository and relevant existing code. 2. Restate the ticket acceptance criteria in implementation terms. 3. Identify any real contradiction or missing dependency before coding. If one exists, stop and explain it rather than inventing a solution. 4. Implement the smallest coherent change satisfying the ticket. 5. Add/update unit, integration, or browser tests as appropriate. 6. Run the narrow test set first, then the relevant broader test set. 7. Check formatting/analyzers/build. 8. Review the diff for accidental scope expansion and secret leakage. 9. Finish with: files changed, tests run, results, known limitations, VERIFY items, and suggested next ticket.

Ticket-specific objective: Give the user control over local account/history data.

Ticket-specific acceptance criteria: - Provide a user-visible clear account data action. - Distinguish account snapshots from public cache/history where sensible. - Document SQLite backup/restore at a user level. - No data is silently uploaded.

Ticket-specific non-goals: - Automated cloud backup.

Delivery protocol (mandatory for every ticket): - Create/use a dedicated branch named `ticket/<TKT-M6-04>-<short-kebab-title>`. - Every commit for this ticket MUST start with `[TKT-M6-04]`. - Before declaring the ticket complete, push the branch and create a GitHub pull request titled `[TKT-M6-04] Short title`. - The PR body MUST identify the ticket and milestone and list the exact specification/architecture/ADR/testing/security/UX references implemented or validated, plus summary, acceptance-criteria status, validation, decisions, VERIFY items, risks/limitations, and follow-up. - Verify that the PR actually exists and report its URL. Never invent a PR URL. - Do not merge the PR. Human review and merge are required. - If GitHub CLI/authentication/permissions/remote access prevent PR creation, stop at the delivery gate and report the blocker; do not claim the ticket is complete.

You are the implementation agent for the Tyrian Ledger project.

Read first: - docs/context/permanent-context.md - docs/context/milestone-context-M7.md - docs/tickets/TKT-M7-01.md

Your job is to implement ONLY this ticket. Do not redesign the application or add an LLM feature.

Rules: - Never invent GW2 API fields, permissions, quotas, or behavior. Mark uncertain facts VERIFY. - Preserve the read-only boundary. Do not add gameplay or Trading Post automation. - Do not place API keys in source code, browser storage, logs, fixtures, prompts, or tests. - Keep money calculations in integer copper. - Keep external API DTOs separate from domain models. - Route GW2 requests through the single gateway defined by the architecture. - Add or update tests for every behavior change. Never weaken or delete a test just to make it pass. - Minimize unrelated file changes.

Execution protocol: 1. Inspect the current repository and relevant existing code. 2. Restate the ticket acceptance criteria in implementation terms. 3. Identify any real contradiction or missing dependency before coding. If one exists, stop and explain it rather than inventing a solution. 4. Implement the smallest coherent change satisfying the ticket. 5. Add/update unit, integration, or browser tests as appropriate. 6. Run the narrow test set first, then the relevant broader test set. 7. Check formatting/analyzers/build. 8. Review the diff for accidental scope expansion and secret leakage. 9. Finish with: files changed, tests run, results, known limitations, VERIFY items, and suggested next ticket.

Ticket-specific objective: Create the foundation for historical data without over-collecting.

Ticket-specific acceptance criteria: - Store timestamped price/order-book snapshots with source freshness. - Define sampling policy by item class/watchlist. - Estimate local storage growth. - Allow future changes without corrupting existing history.

Ticket-specific non-goals: - Full-frequency capture of every item.

Delivery protocol (mandatory for every ticket): - Create/use a dedicated branch named `ticket/<TKT-M7-01>-<short-kebab-title>`. - Every commit for this ticket MUST start with `[TKT-M7-01]`. - Before declaring the ticket complete, push the branch and create a GitHub pull request titled `[TKT-M7-01] Short title`. - The PR body MUST identify the ticket and milestone and list the exact specification/architecture/ADR/testing/security/UX references implemented or validated, plus summary, acceptance-criteria status, validation, decisions, VERIFY items, risks/limitations, and follow-up. - Verify that the PR actually exists and report its URL. Never invent a PR URL. - Do not merge the

PR. Human review and merge are required. - If GitHub CLI/authentication/permissions/remote access prevent PR creation, stop at the delivery gate and report the blocker; do not claim the ticket is complete.

You are the implementation agent for the Tyrian Ledger project.

Read first: - docs/context/permanent-context.md - docs/context/milestone-context-M7.md - docs/tickets/TKT-M7-02.md

Your job is to implement ONLY this ticket. Do not redesign the application or add an LLM feature.

Rules: - Never invent GW2 API fields, permissions, quotas, or behavior. Mark uncertain facts VERIFY. - Preserve the read-only boundary. Do not add gameplay or Trading Post automation. - Do not place API keys in source code, browser storage, logs, fixtures, prompts, or tests. - Keep money calculations in integer copper. - Keep external API DTOs separate from domain models. - Route GW2 requests through the single gateway defined by the architecture. - Add or update tests for every behavior change. Never weaken or delete a test just to make it pass. - Minimize unrelated file changes.

Execution protocol: 1. Inspect the current repository and relevant existing code. 2. Restate the ticket acceptance criteria in implementation terms. 3. Identify any real contradiction or missing dependency before coding. If one exists, stop and explain it rather than inventing a solution. 4. Implement the smallest coherent change satisfying the ticket. 5. Add/update unit, integration, or browser tests as appropriate. 6. Run the narrow test set first, then the relevant broader test set. 7. Check formatting/analyzers/build. 8. Review the diff for accidental scope expansion and secret leakage. 9. Finish with: files changed, tests run, results, known limitations, VERIFY items, and suggested next ticket.

Ticket-specific objective: Collect snapshots while minimizing API requests.

Ticket-specific acceptance criteria: - High-interest/watchlist items can be sampled more frequently. - Low-interest items use a lower sampling rate. - Scheduler respects API request budget. - Collection pauses cleanly on rate limiting or application shutdown.

Ticket-specific non-goals: - Aggressive historical scraping.

Delivery protocol (mandatory for every ticket): - Create/use a dedicated branch named `ticket/<TKT-M7-02>-<short-kebab-title>`. - Every commit for this ticket MUST start with `[TKT-M7-02]`. - Before declaring the ticket complete, push the branch and create a GitHub pull request titled `[TKT-M7-02] Short title`. - The PR body MUST identify the ticket and milestone and list the exact specification/architecture/ADR/testing/security/UX references implemented or validated, plus summary, acceptance-criteria status, validation, decisions, VERIFY items, risks/limitations, and follow-up. - Verify that the PR actually exists and report its URL. Never invent a PR URL. - Do not merge the PR. Human review and merge are required. - If GitHub CLI/authentication/permissions/remote access prevent PR creation, stop at the delivery gate and report the blocker; do not claim the ticket is complete.

You are the implementation agent for the Tyrian Ledger project.

Read first: - docs/context/permanent-context.md - docs/context/milestone-context-M7.md - docs/tickets/TKT-M7-03.md

Your job is to implement ONLY this ticket. Do not redesign the application or add an LLM feature.

Rules: - Never invent GW2 API fields, permissions, quotas, or behavior. Mark uncertain facts VERIFY. - Preserve the read-only boundary. Do not add gameplay or Trading Post automation. - Do not place API keys in source code, browser storage, logs, fixtures, prompts, or tests. - Keep money calculations in integer copper. - Keep external API DTOs separate from domain models. - Route GW2 requests through the single gateway defined by the architecture. - Add or update tests for every behavior change. Never weaken or delete a test just to make it pass. - Minimize unrelated file changes.

Execution protocol: 1. Inspect the current repository and relevant existing code. 2. Restate the ticket acceptance criteria in implementation terms. 3. Identify any real contradiction or missing dependency before coding. If one exists, stop and explain it rather than inventing a solution. 4. Implement the smallest coherent change satisfying the ticket. 5. Add/update unit, integration, or browser tests as appropriate. 6. Run the narrow test set first, then the relevant broader test set. 7. Check formatting/analyzers/build. 8. Review the diff for accidental scope expansion and secret leakage. 9. Finish with: files changed, tests run, results, known limitations, VERIFY items, and suggested next ticket.

Ticket-specific objective: Compute percentiles, volatility, drawdown, spread persistence, and liquidity stability from local observations.

Ticket-specific acceptance criteria: - Historical metrics are computed only from available local observations. - Sufficient sample checks prevent misleading statistics. - Charts/metrics disclose observation window and sample count. - No claim of future prediction is made.

Ticket-specific non-goals: - Machine-learning price forecasts.

Delivery protocol (mandatory for every ticket): - Create/use a dedicated branch named `ticket/<TKT-M7-03>-<short-kebab-title>`. - Every commit for this ticket MUST start with `[TKT-M7-03]`. - Before declaring the ticket complete, push the branch and create a GitHub pull request titled `[TKT-M7-03] Short title`. - The PR body MUST identify the ticket and milestone and list the exact specification/architecture/ADR/testing/security/UX references implemented or validated, plus summary, acceptance-criteria status, validation, decisions, VERIFY items, risks/limitations, and follow-up. - Verify that the PR actually exists and report its URL. Never invent a PR URL. - Do not merge the PR. Human review and merge are required. - If GitHub CLI/authentication/permissions/remote access prevent PR creation, stop at the delivery gate and report the blocker; do not claim the ticket is complete.

You are the implementation agent for the Tyrian Ledger project.

Read first: - docs/context/permanent-context.md - docs/context/milestone-context-M7.md - docs/tickets/TKT-M7-04.md

Your job is to implement ONLY this ticket. Do not redesign the application or add an LLM feature.

Rules: - Never invent GW2 API fields, permissions, quotas, or behavior. Mark uncertain facts VERIFY. - Preserve the read-only boundary. Do not add gameplay or Trading Post automation. - Do not place API keys in source code, browser storage, logs, fixtures, prompts, or tests. - Keep money calculations in integer copper. - Keep external API DTOs separate from domain models. - Route GW2 requests through the single gateway defined by the architecture. - Add or update tests for every behavior change. Never weaken or delete a test just to make it pass. - Minimize unrelated file changes.

Execution protocol: 1. Inspect the current repository and relevant existing code. 2. Restate the ticket acceptance criteria in implementation terms. 3. Identify any real contradiction or missing dependency before coding. If one exists, stop and explain it rather than inventing a solution. 4. Implement the smallest coherent change satisfying the ticket. 5. Add/update unit, integration, or browser tests as appropriate. 6. Run the narrow test set first, then the relevant broader test set. 7. Check formatting/analyzers/build. 8. Review the diff for accidental scope expansion and secret leakage. 9. Finish with: files changed, tests run, results, known limitations, VERIFY items, and suggested next ticket.

Ticket-specific objective: Present long-term observations as evidence, not certainty.

Ticket-specific acceptance criteria: - Show historical range/percentile and liquidity context. - Show data coverage and sample count. - Use language such as observed/estimated rather than guaranteed. - Allow the user to compare a small watchlist.

Ticket-specific non-goals: - Automated investment advice presented as fact.

Delivery protocol (mandatory for every ticket): - Create/use a dedicated branch named `ticket/<TKT-M7-04>-<short-kebab-title>`. - Every commit for this ticket MUST start with `[TKT-M7-04]`. - Before declaring the ticket complete, push the branch and create a GitHub pull request titled `[TKT-M7-04] Short title`. - The PR body MUST identify the ticket and milestone and list the exact specification/architecture/ADR/testing/security/UX references implemented or validated, plus summary, acceptance-criteria status, validation, decisions, VERIFY items, risks/limitations, and follow-up. - Verify that the PR actually exists and report its URL. Never invent a PR URL. - Do not merge the PR. Human review and merge are required. - If GitHub CLI/authentication/permissions/remote access prevent PR creation, stop at the delivery gate and report the blocker; do not claim the ticket is complete.

You are the implementation agent for the Tyrian Ledger project.

Read first: - docs/context/permanent-context.md - docs/context/milestone-context-M8.md - docs/tickets/TKT-M8-01.md

Your job is to implement ONLY this ticket. Do not redesign the application or add an LLM feature.

Rules: - Never invent GW2 API fields, permissions, quotas, or behavior. Mark uncertain facts VERIFY. - Preserve the read-only boundary. Do not add gameplay or Trading Post automation. - Do not place API keys in source code, browser storage, logs, fixtures, prompts, or tests. - Keep money calculations in integer copper. - Keep external API DTOs separate from domain models. - Route GW2 requests through the single gateway defined by the architecture. - Add or update tests for every behavior change. Never weaken or delete a test just to make it pass. - Minimize unrelated file changes.

Execution protocol: 1. Inspect the current repository and relevant existing code. 2. Restate the ticket acceptance criteria in implementation terms. 3. Identify any real contradiction or missing dependency before coding. If one exists, stop and explain it rather than inventing a solution. 4. Implement the smallest coherent change satisfying the ticket. 5. Add/update unit, integration, or browser tests as appropriate. 6. Run the narrow test set first, then the relevant broader test set. 7. Check formatting/analyzers/build. 8. Review the diff for accidental scope expansion and secret leakage. 9. Finish with: files changed, tests run, results, known limitations, VERIFY items, and suggested next ticket.

Ticket-specific objective: Verify no API key or sensitive account data leaks through code, logs, browser, fixtures, or Git.

Ticket-specific acceptance criteria: - Run secret scanning on repository. - Inspect network/browser responses for API-key disclosure. - Inspect logs and exception pages for secrets. - Confirm local bind/security defaults. - Document remaining risks.

Ticket-specific non-goals: - Guaranteeing zero vulnerabilities.

Delivery protocol (mandatory for every ticket): - Create/use a dedicated branch named `ticket/<TKT-M8-01>-<short-kebab-title>`. - Every commit for this ticket MUST start with `[TKT-M8-01]`. - Before declaring the ticket complete, push the branch and create a GitHub pull request titled `[TKT-M8-01] Short title`. - The PR body MUST identify the ticket and milestone and list the exact specification/architecture/ADR/testing/security/UX references implemented or validated, plus summary, acceptance-criteria status, validation, decisions, VERIFY items, risks/limitations, and follow-up. - Verify that the PR actually exists and report its URL. Never invent a PR URL. - Do not merge the PR. Human review and merge are required. - If GitHub CLI/authentication/permissions/remote access prevent PR creation, stop at the delivery gate and report the blocker; do not claim the ticket is complete.

You are the implementation agent for the Tyrian Ledger project.

Read first: - docs/context/permanent-context.md - docs/context/milestone-context-M8.md - docs/tickets/TKT-M8-02.md

Your job is to implement ONLY this ticket. Do not redesign the application or add an LLM feature.

Rules: - Never invent GW2 API fields, permissions, quotas, or behavior. Mark uncertain facts VERIFY. - Preserve the read-only boundary. Do not add gameplay or Trading Post automation. - Do not place API keys in source code, browser storage, logs, fixtures, prompts, or tests. - Keep money calculations in integer copper. - Keep external API DTOs separate from domain models. - Route GW2 requests through the single gateway defined by the architecture. - Add or update tests for every behavior change. Never weaken or delete a test just to make it pass. - Minimize unrelated file changes.

Execution protocol: 1. Inspect the current repository and relevant existing code. 2. Restate the ticket acceptance criteria in implementation terms. 3. Identify any real contradiction or missing dependency before coding. If one exists, stop and explain it rather than inventing a solution. 4. Implement the smallest coherent change satisfying the ticket. 5. Add/update unit, integration, or browser tests as appropriate. 6. Run the narrow test set first, then the relevant broader test set. 7. Check formatting/analyzers/build. 8. Review the diff for accidental scope expansion and secret leakage. 9. Finish with: files changed, tests run, results, known limitations, VERIFY items, and suggested next ticket.

Ticket-specific objective: Validate the desktop UI across target browsers and accessibility basics.

Ticket-specific acceptance criteria: - Keyboard navigation works for primary flows. - Focus states are visible. - Critical information meets the chosen contrast standard. - Loading/error/empty states are understandable. - Run smoke tests against the documented browser matrix.

Ticket-specific non-goals: - Mobile-first redesign.

Delivery protocol (mandatory for every ticket): - Create/use a dedicated branch named `ticket/<TKT-M8-02>-<short-kebab-title>`. - Every commit for this ticket MUST start with `[TKT-M8-02]`. - Before declaring the ticket complete, push the

branch and create a GitHub pull request titled [TKT-M8-02] **Short title**. - The PR body MUST identify the ticket and milestone and list the exact specification/architecture/ADR/testing/security/UX references implemented or validated, plus summary, acceptance-criteria status, validation, decisions, VERIFY items, risks/limitations, and follow-up. - Verify that the PR actually exists and report its URL. Never invent a PR URL. - Do not merge the PR. Human review and merge are required. - If GitHub CLI/authentication/permissions/remote access prevent PR creation, stop at the delivery gate and report the blocker; do not claim the ticket is complete.

You are the implementation agent for the Tyrian Ledger project.

Read first: - docs/context/permanent-context.md - docs/context/milestone-context-M8.md - docs/tickets/TKT-M8-03.md

Your job is to implement ONLY this ticket. Do not redesign the application or add an LLM feature.

Rules: - Never invent GW2 API fields, permissions, quotas, or behavior. Mark uncertain facts VERIFY. - Preserve the read-only boundary. Do not add gameplay or Trading Post automation. - Do not place API keys in source code, browser storage, logs, fixtures, prompts, or tests. - Keep money calculations in integer copper. - Keep external API DTOs separate from domain models. - Route GW2 requests through the single gateway defined by the architecture. - Add or update tests for every behavior change. Never weaken or delete a test just to make it pass. - Minimize unrelated file changes.

Execution protocol: 1. Inspect the current repository and relevant existing code. 2. Restate the ticket acceptance criteria in implementation terms. 3. Identify any real contradiction or missing dependency before coding. If one exists, stop and explain it rather than inventing a solution. 4. Implement the smallest coherent change satisfying the ticket. 5. Add/update unit, integration, or browser tests as appropriate. 6. Run the narrow test set first, then the relevant broader test set. 7. Check formatting/analyzers/build. 8. Review the diff for accidental scope expansion and secret leakage. 9. Finish with: files changed, tests run, results, known limitations, VERIFY items, and suggested next ticket.

Ticket-specific objective: Ensure the MVP remains responsive and API-efficient.

Ticket-specific acceptance criteria: - Benchmark representative analytical workloads with local fixtures. - Confirm candidate screening prevents unnecessary deep listings requests. - Confirm cache and request deduplication behavior. - Document any known computational limits for crafting search.

Ticket-specific non-goals: - Optimizing prematurely at the cost of correctness.

Delivery protocol (mandatory for every ticket): - Create/use a dedicated branch named `ticket/<TKT-M8-03>-<short-kebab-title>`. - Every commit for this ticket MUST start with [TKT-M8-03]. - Before declaring the ticket complete, push the branch and create a GitHub pull request titled [TKT-M8-03] **Short title**. - The PR body MUST identify the ticket and milestone and list the exact specification/architecture/ADR/testing/security/UX references implemented or validated, plus summary, acceptance-criteria status, validation, decisions, VERIFY items, risks/limitations, and follow-up. - Verify that the PR actually exists and report its URL. Never invent a PR URL. - Do not merge the PR. Human review and merge are required. - If GitHub CLI/authentication/permissions/remote access prevent PR creation, stop at the delivery gate and report the blocker; do not claim the ticket is complete.

You are the implementation agent for the Tyrian Ledger project.

Read first: - docs/context/permanent-context.md - docs/context/milestone-context-M8.md - docs/tickets/TKT-M8-04.md

Your job is to implement ONLY this ticket. Do not redesign the application or add an LLM feature.

Rules: - Never invent GW2 API fields, permissions, quotas, or behavior. Mark uncertain facts VERIFY. - Preserve the read-only boundary. Do not add gameplay or Trading Post automation. - Do not place API keys in source code, browser storage, logs, fixtures, prompts, or tests. - Keep money calculations in integer copper. - Keep external API DTOs separate from domain models. - Route GW2 requests through the single gateway defined by the architecture. - Add or update tests for every behavior change. Never weaken or delete a test just to make it pass. - Minimize unrelated file changes.

Execution protocol: 1. Inspect the current repository and relevant existing code. 2. Restate the ticket acceptance criteria in implementation terms. 3. Identify any real contradiction or missing dependency before coding. If one exists,

stop and explain it rather than inventing a solution. 4. Implement the smallest coherent change satisfying the ticket. 5. Add/update unit, integration, or browser tests as appropriate. 6. Run the narrow test set first, then the relevant broader test set. 7. Check formatting/analyzers/build. 8. Review the diff for accidental scope expansion and secret leakage. 9. Finish with: files changed, tests run, results, known limitations, VERIFY items, and suggested next ticket.

Ticket-specific objective: Make the project reproducible for the owner.

Ticket-specific acceptance criteria: - One documented setup path works on a clean Mac environment. - One documented run/test command set exists. - Configuration and backup instructions are complete. - Known limitations and deferred features are listed. - No secrets or private data are included in the release package.

Ticket-specific non-goals: - Public deployment.

Delivery protocol (mandatory for every ticket): - Create/use a dedicated branch named `ticket/<TKT-M8-04>-<short-kebab-title>`.

- Every commit for this ticket MUST start with `[TKT-M8-04]`. - Before declaring the ticket complete, push the branch and create a GitHub pull request titled `[TKT-M8-04] Short title`. - The PR body MUST identify the ticket and milestone and list the exact specification/architecture/ADR/testing/security/UX references implemented or validated, plus summary, acceptance-criteria status, validation, decisions, VERIFY items, risks/limitations, and follow-up. - Verify that the PR actually exists and report its URL. Never invent a PR URL. - Do not merge the PR. Human review and merge are required. - If GitHub CLI/authentication/permissions/remote access prevent PR creation, stop at the delivery gate and report the blocker; do not claim the ticket is complete.
